

# Linux Project

## Project Summary :::

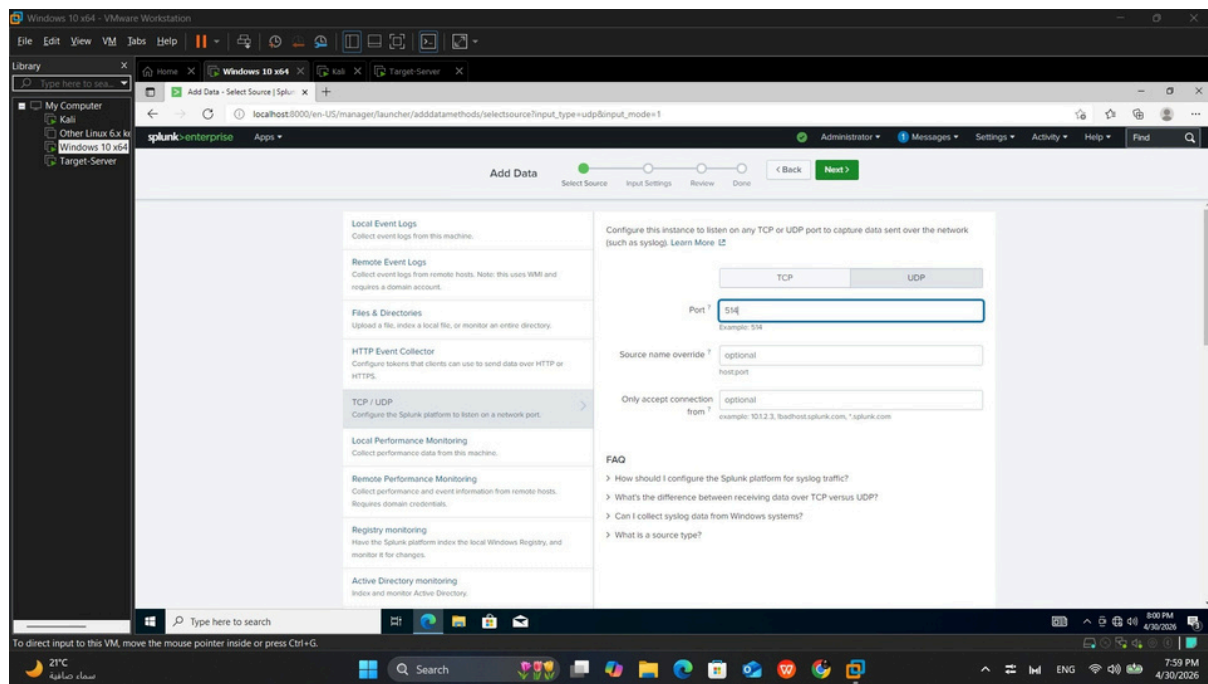
**Linux Project v10** is a comprehensive simulation of a real-world cyber attack and defense lifecycle, executed within a controlled laboratory environment. The project aims to bridge the gap between theoretical security concepts and practical application by establishing a multi-layered infrastructure consisting of a vulnerable web application (Target), an offensive workstation (Attacker), and a Security Information and Event Management (SIEM) solution for monitoring and detection.

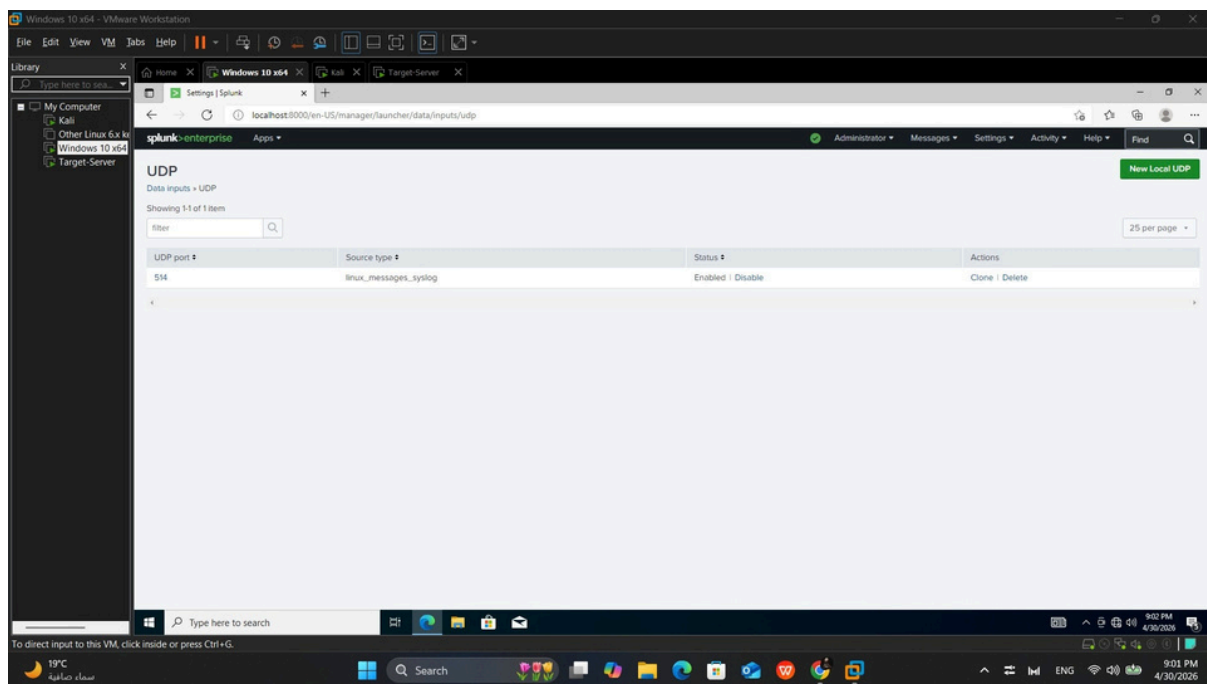
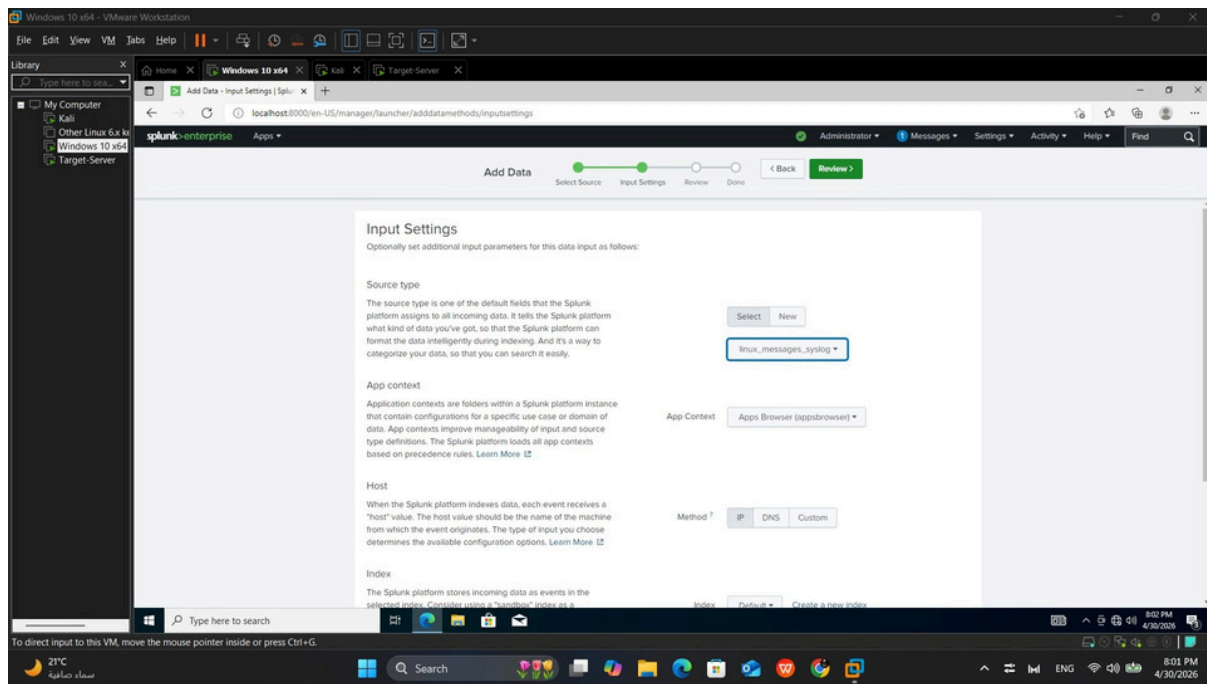
The project is structured into four specialized domains to ensure a holistic approach to cybersecurity:

1. **Architecture & Visibility:** Deployment of the vulnerable environment and establishing robust log forwarding mechanisms (Apache and Bash history) to ensure full observability.
2. **Offensive Operations (Red Teaming):** Conducting a black-box penetration test to identify and exploit critical web vulnerabilities, such as **XSS, OS Command Injection, and File Upload**, with the ultimate goal of achieving Remote Code Execution (RCE).
3. **Defensive Operations (Blue Teaming & IR):** Reconstructing the attack timeline through the SIEM, performing incident investigation, and developing custom detection queries to identify malicious activities in real-time.
4. **Mitigation & Remediation (DevSecOps):** Implementing code-level fixes and utilizing Version Control (**Git**) to manage security patches, followed by re-exploitation testing to verify the effectiveness of the applied defenses.

By integrating these four pillars, **Linux Project V10** provides a rigorous framework for understanding how vulnerabilities are exploited, detected, and remediated in modern enterprise networks, ultimately strengthening the security posture against sophisticated cyber threats.

"As part of the foundational infrastructure setup, I deployed and configured the primary virtual machines required for the project. An **Ubuntu Linux** system was established as the **Target Machine**, hosting the vulnerable web application to serve as the focal point for the attack simulations. Simultaneously, a **Windows-based machine** was configured to host the **SIEM solution**, serving as the central security monitoring hub. This setup ensured the seamless ingestion, correlation, and analysis of logs and security events forwarded from the Ubuntu target, providing full visibility across the environment."

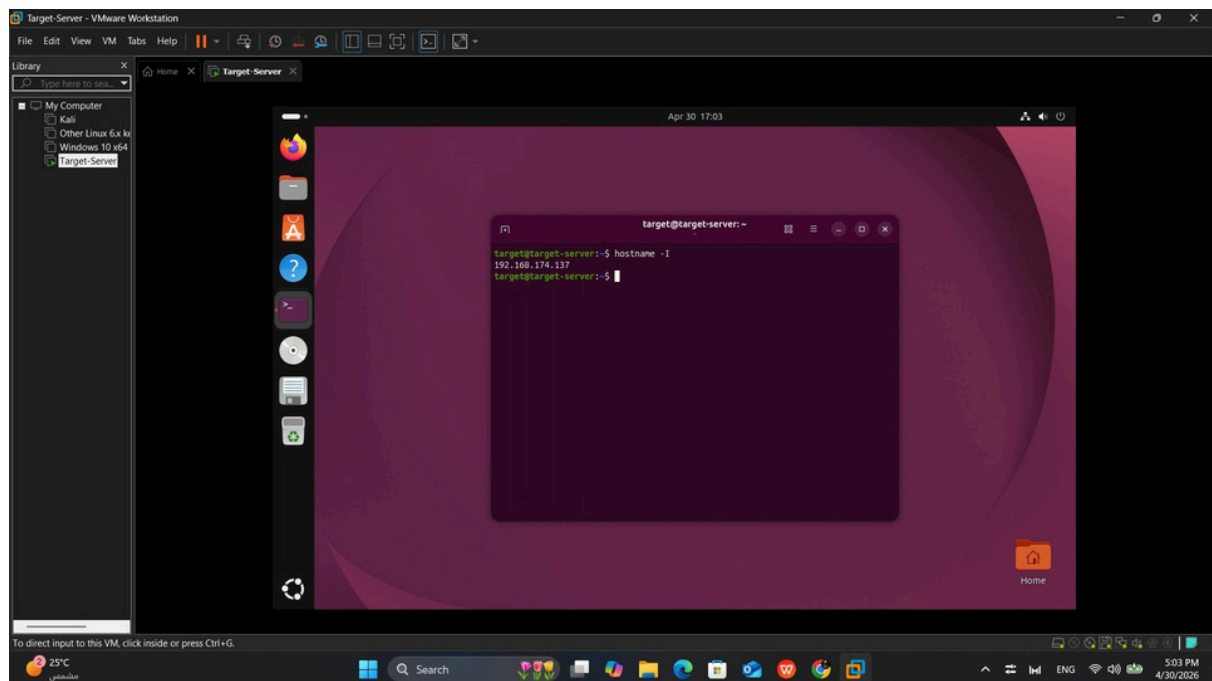
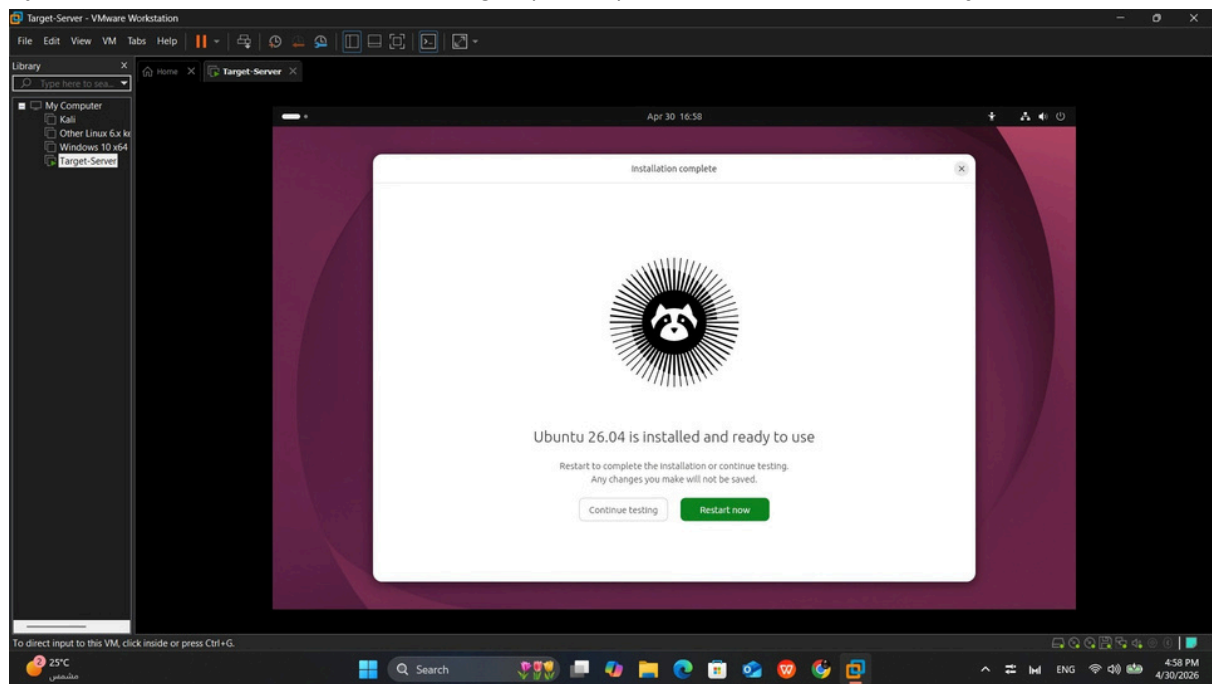




The **Splunk Enterprise** platform was utilized as the core SIEM solution on the Windows machine. As shown in the configuration, a **UDP data input** was established on **Port 514** to ingest Syslog data. This configuration enables the centralized collection and indexing of logs from the Ubuntu target, providing the necessary visibility for real-time monitoring and incident investigation."

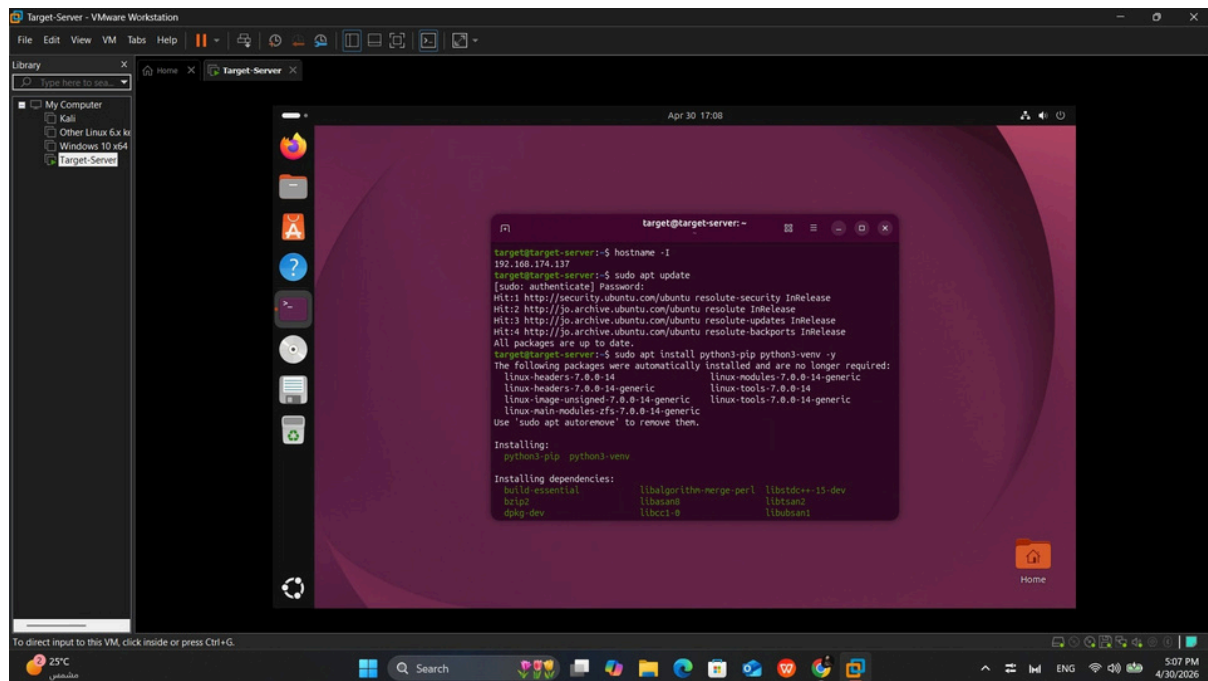
=====

In this phase, I specifically handled the deployment and preparation of the Ubuntu Operating System, which functioned as the Target (Victim) Machine for the entire project simulation."



In this stage, I verified the network configuration and IP addressing for the **Ubuntu Target Machine**. Using the `hostname -I` command, I confirmed that the machine is active on the network with the internal IP address **192.168.174.137**

=====

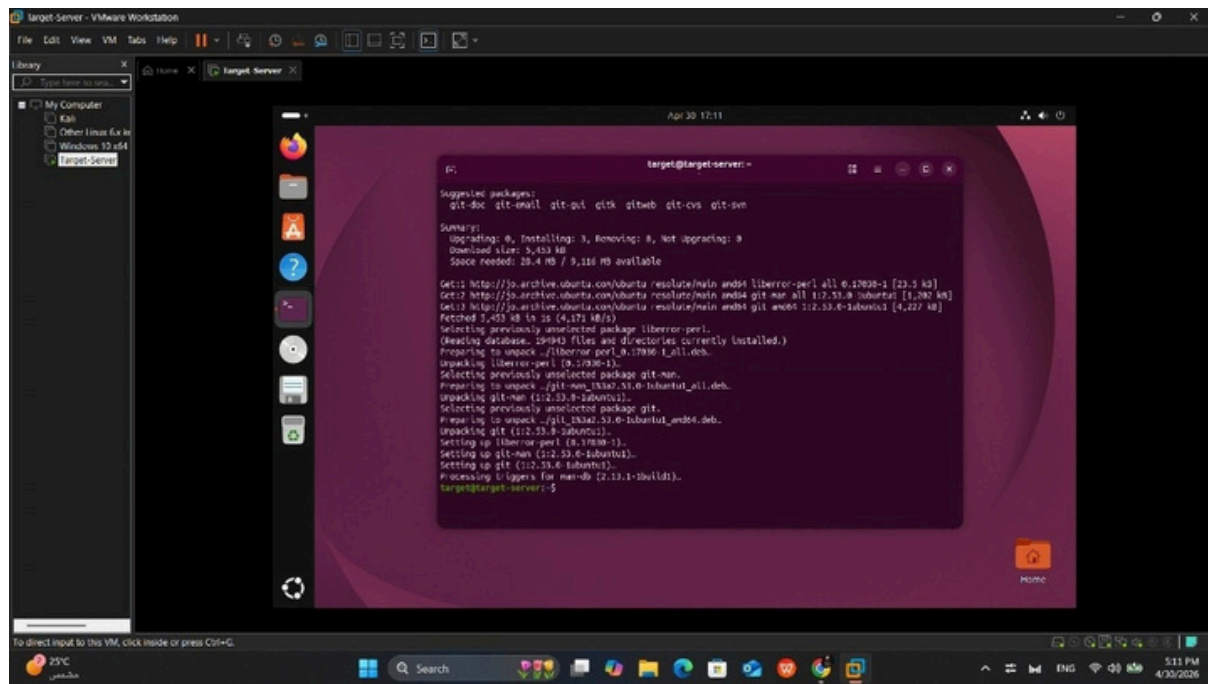


After verifying the network settings, I proceeded to prepare the software environment on the **Ubuntu Target Machine**. This step is crucial for hosting the web application and its dependencies."

#### **Actions performed in this stage:**

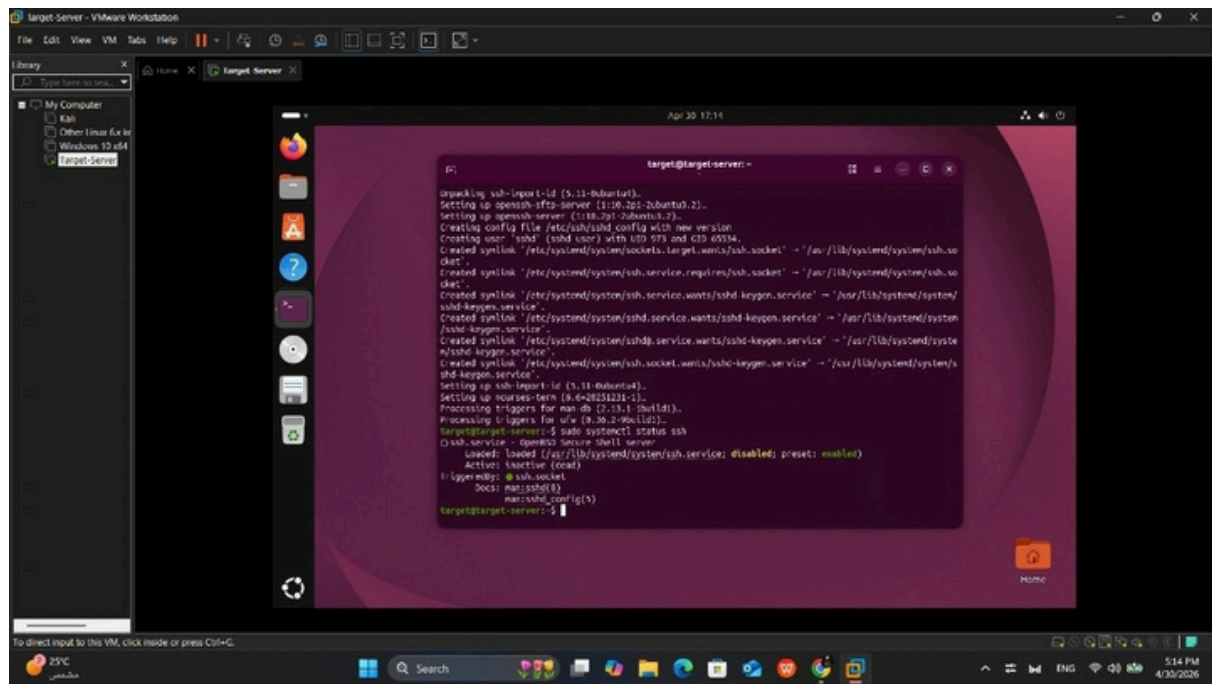
- **System Update:** Executed `sudo apt update` to synchronize the package index files from their sources, ensuring system stability.
- **Development Dependencies:** Installed **Python3-pip** and **Python3-venv**. These tools are essential for managing the Python environment and installing the libraries required to run the vulnerable web application (such as Flask or Django).
- **Automation:** Used the `-y` flag to automate the installation process, ensuring all necessary dependencies were met without manual interruption

=====



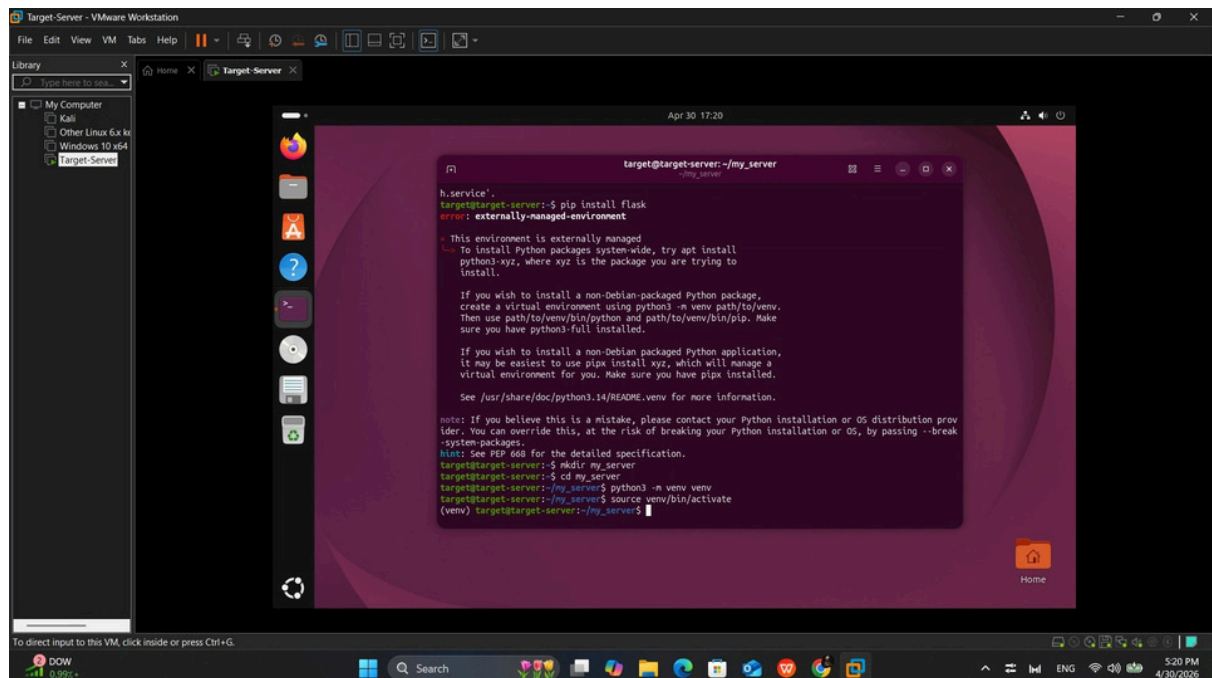
## Section: Installation of Version Control System (Git)

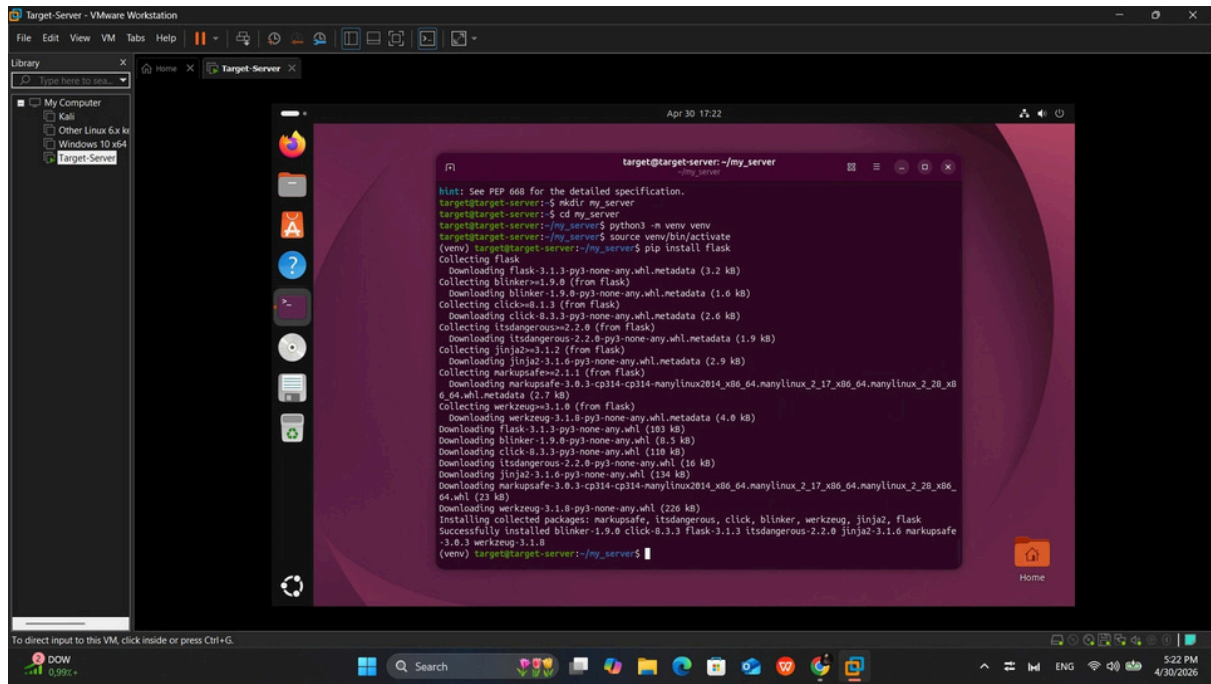
"To facilitate collaboration and track code changes throughout the project lifecycle, I installed **Git** on the Ubuntu Target Machine. This setup is essential for the **DevSecOps** workflow integrated into our project."



## Section: SSH Service Configuration and Verification

"To ensure remote management capabilities and facilitate the incident response phase, I installed and configured the **OpenSSH Server** on the Ubuntu Target Machine."

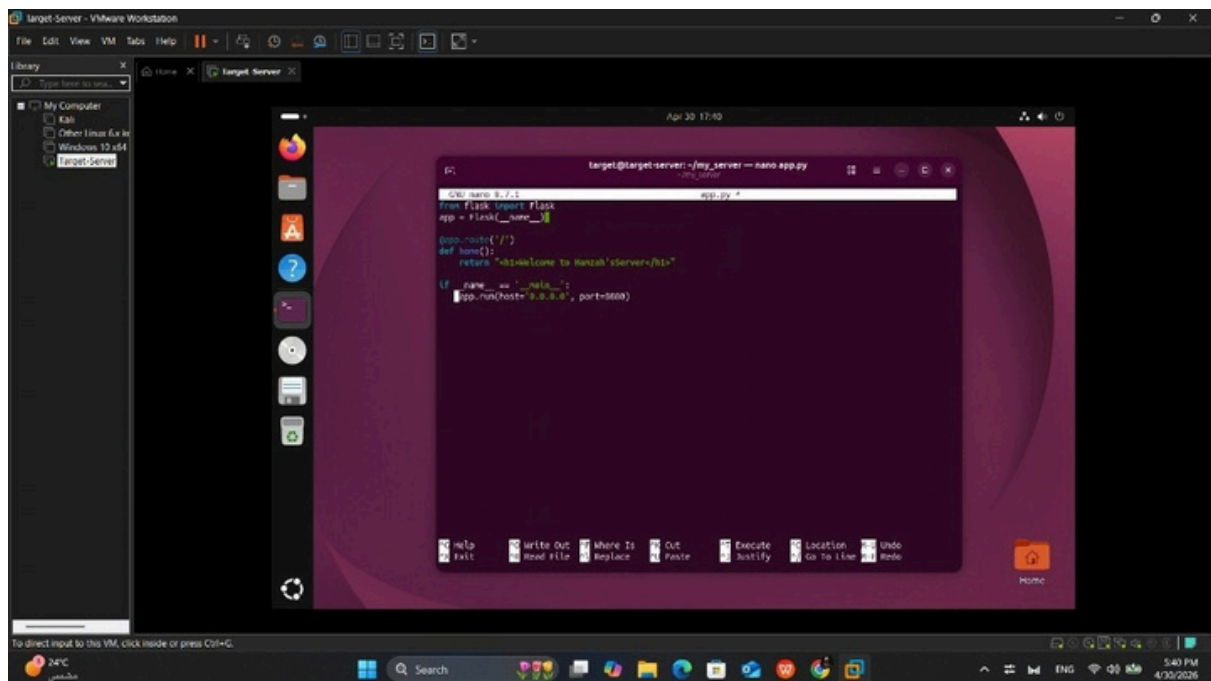




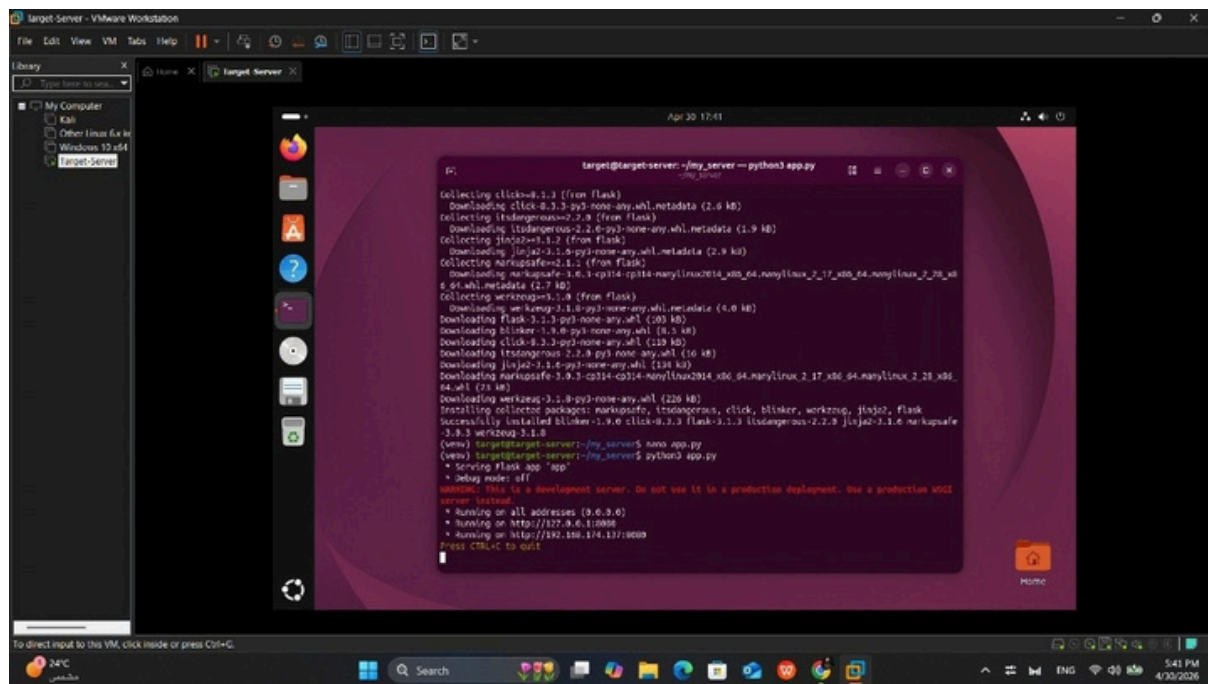
## Section: Web Application Framework Installation (Flask)

"To ensure a stable and isolated environment for the vulnerable web application, I utilized a **Python Virtual Environment (Venv)**. This prevents dependency conflicts and adheres to Linux best practices for software deployment

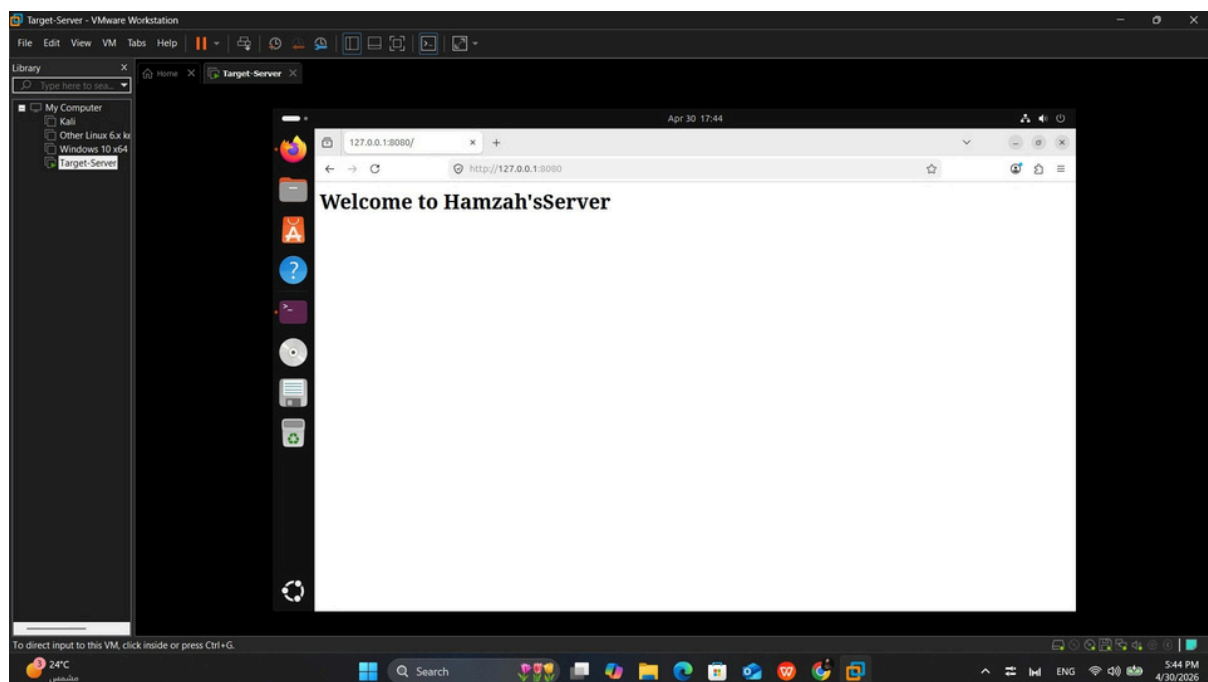
1::



2::



3::



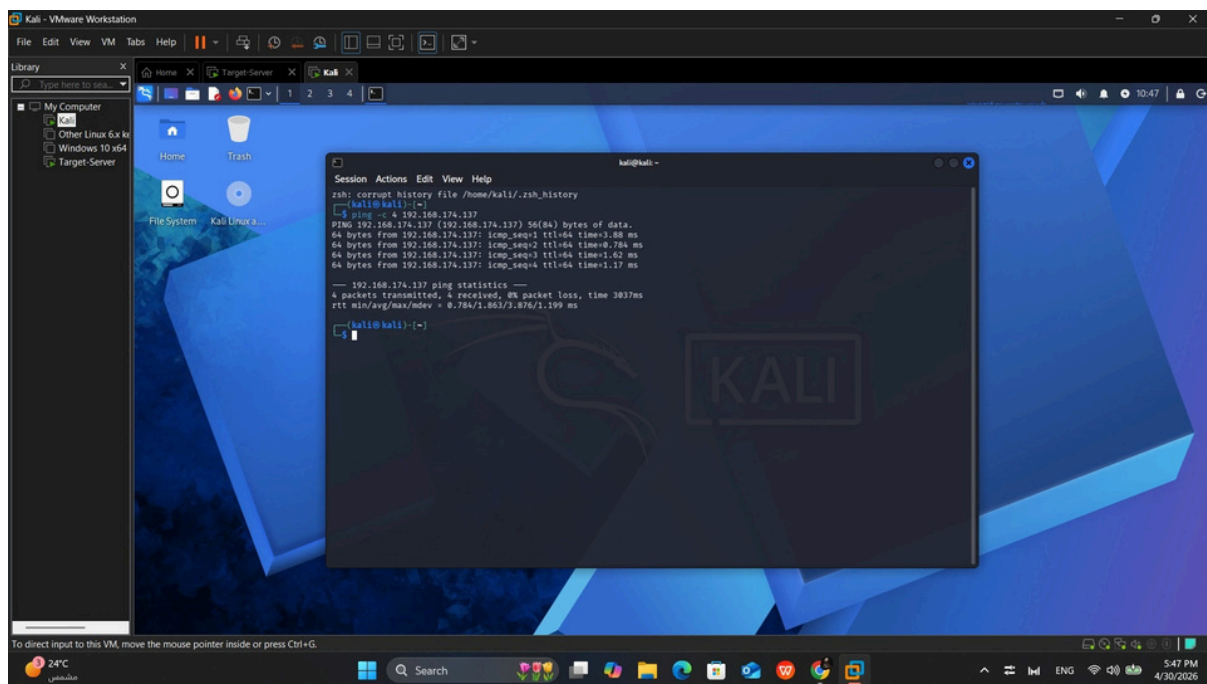
## Section: Web Application Development and Deployment

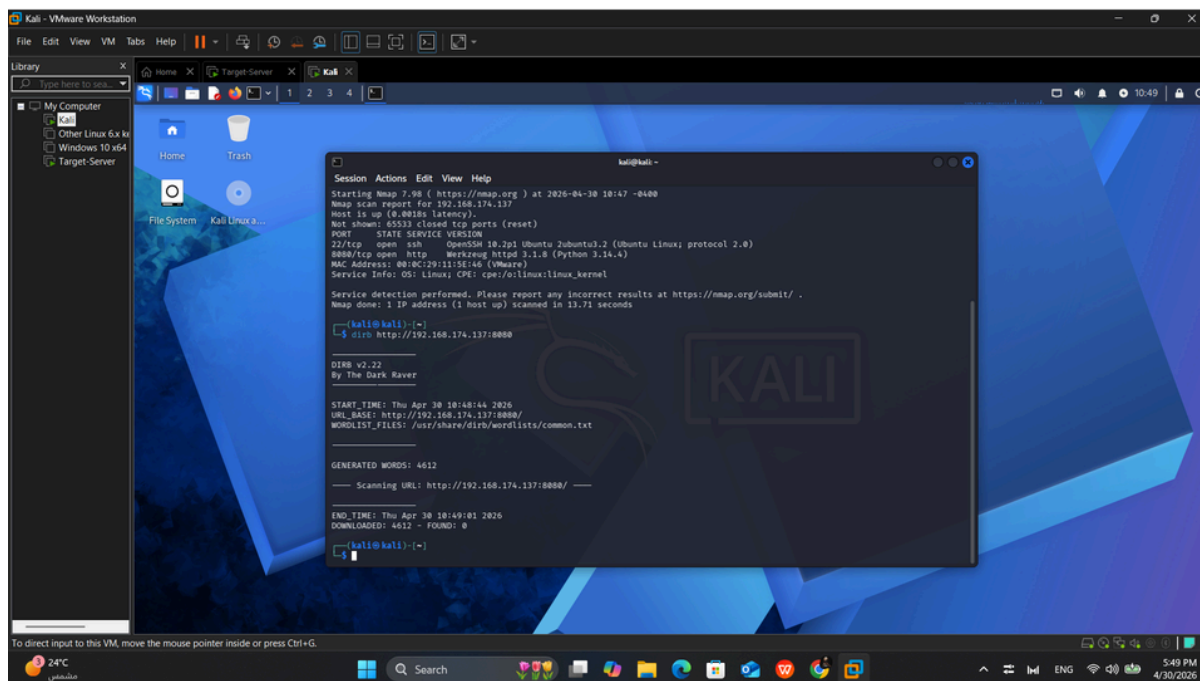
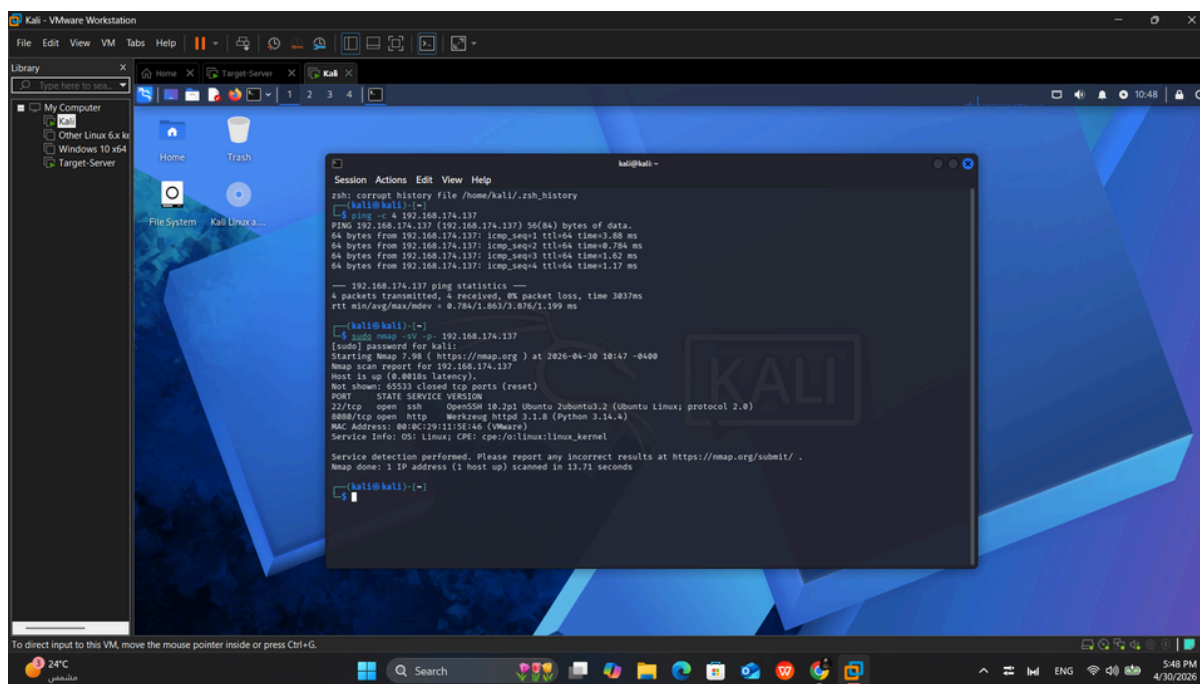
"After preparing the environment, I proceeded to develop and launch the baseline web application. This phase confirms that the server is operational and ready for the subsequent integration of security vulnerabilities."

**Breakdown of the steps shown in the screenshots:**

1. **Code Implementation (Image 1):**
  - Using the `nano` text editor, I created `app.py`.
  - I wrote a Flask-based script to initialize the web server.
  - Notably, I configured the application to run on `host='0.0.0.0'` and `port=8080`, ensuring the server is accessible from other machines in the network (Attacker and SIEM) rather than just the local loopback.
2. **Server Execution (Image 2):**
  - I executed the command `python3 app.py` within the virtual environment.
  - The terminal output confirms that the Flask development server is active and listening for incoming requests on the assigned IP address (`192.168.174.137:8080`).
3. **Deployment Verification (Image 3):**
  - I verified the application's status by accessing it via the Firefox browser at `http://127.0.0.1:8080`.
  - The successful rendering of the message **"Welcome to Hamzah'sServer"** confirms that the architecture setup is complete and the application is functioning correctly.

## Section 2: Offensive Operations





## Section 3: Reconnaissance and Information Gathering

"Once the infrastructure was fully deployed, I initiated the reconnaissance phase from the **Kali Linux Attacker machine** to identify active services and potential entry points on the target system."

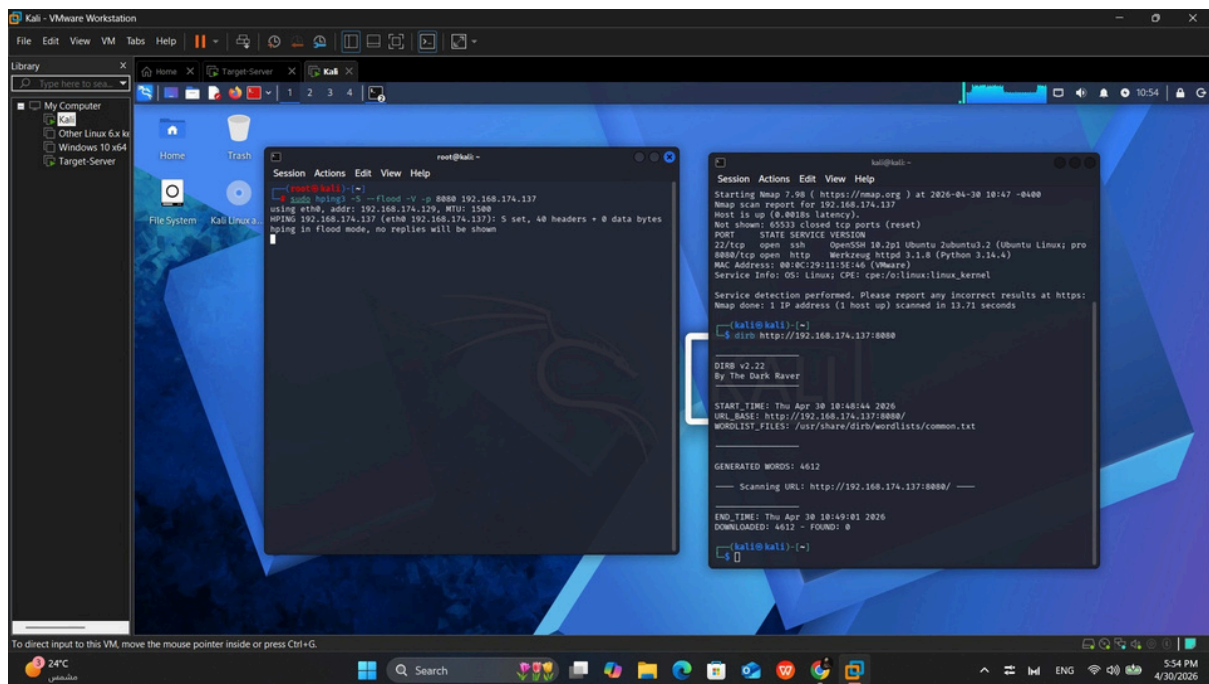
**Analysis of the performed steps:**

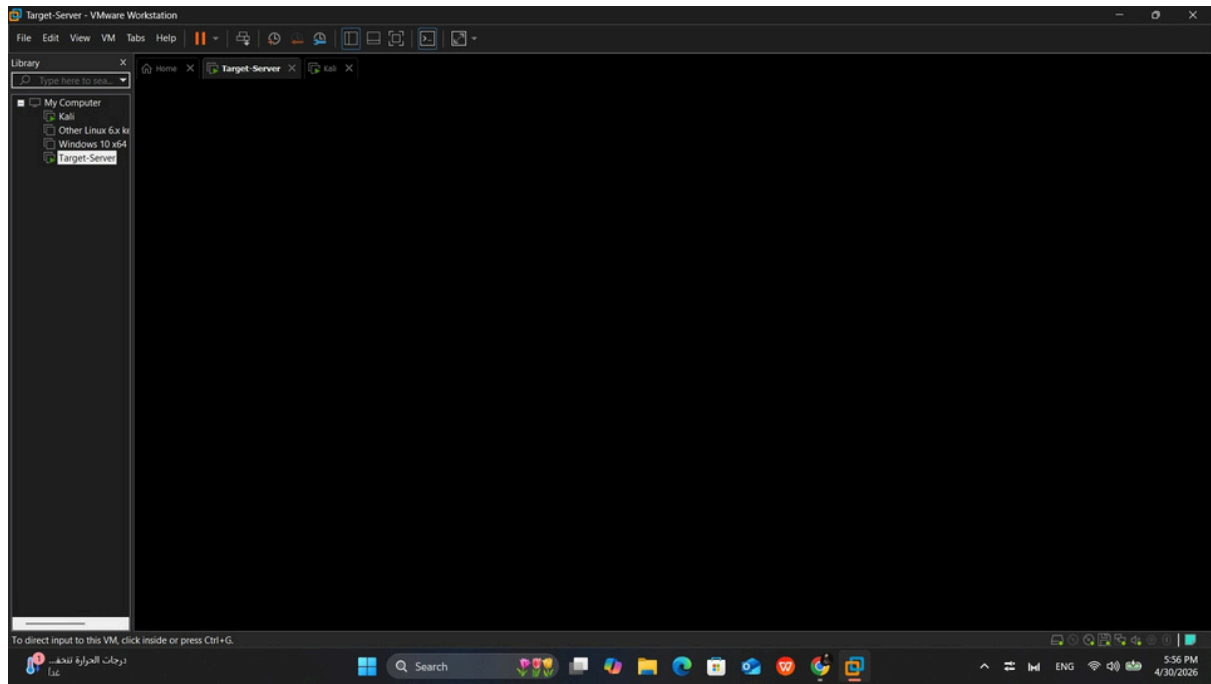
### 1. Network Connectivity Verification (Image 2 - Ping):

- I executed the `ping -c 4 192.168.174.137` command.

- **Result:** 0% packet loss, confirming that the Ubuntu Target is reachable over the network and responding to ICMP requests.
- 2. **Service and Port Scanning (Image 3 - Nmap):**
  - I performed an advanced scan using `sudo nmap -sV -p- 192.168.174.137`.
  - **Findings:**
    - **Port 22/tcp:** Open (SSH), running OpenSSH 10.2p1.
    - **Port 8080/tcp:** Open (HTTP), identified as the **Werkzeug httpd 3.1.8 (Python 3.14.4)** server.
  - This scan confirms that the Flask web application is live and accessible on port 8080.
- 3. **Directory Brute-Forcing (Image 4 - DIRB):**
  - I used the `dirb` tool against `http://192.168.174.137:8080/` to discover hidden directories or files.
  - This is a crucial step in **Vulnerability Assessment** to find sensitive paths that might lead to exploitation

=====





## Section: Denial of Service (DoS) Attack Simulation

"To test the resilience of the Ubuntu Target Machine and the detection capabilities of the SIEM, I conducted a **SYN Floodattack** , which is a form of Denial of Service (DoS)."

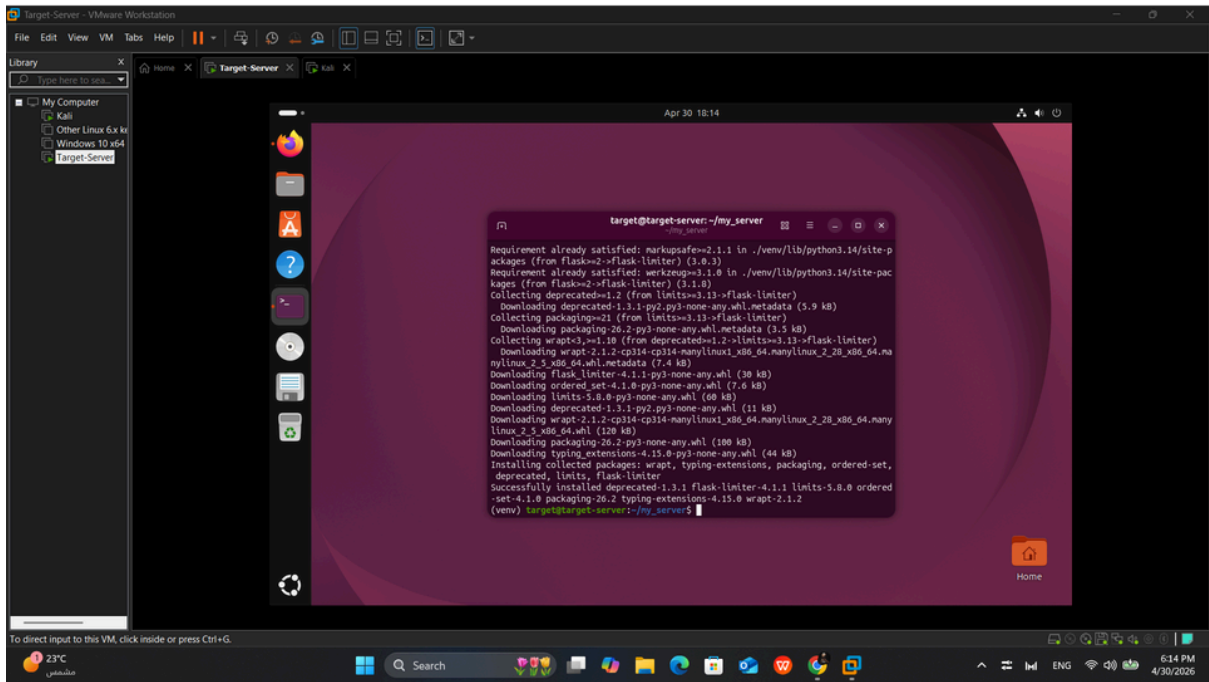
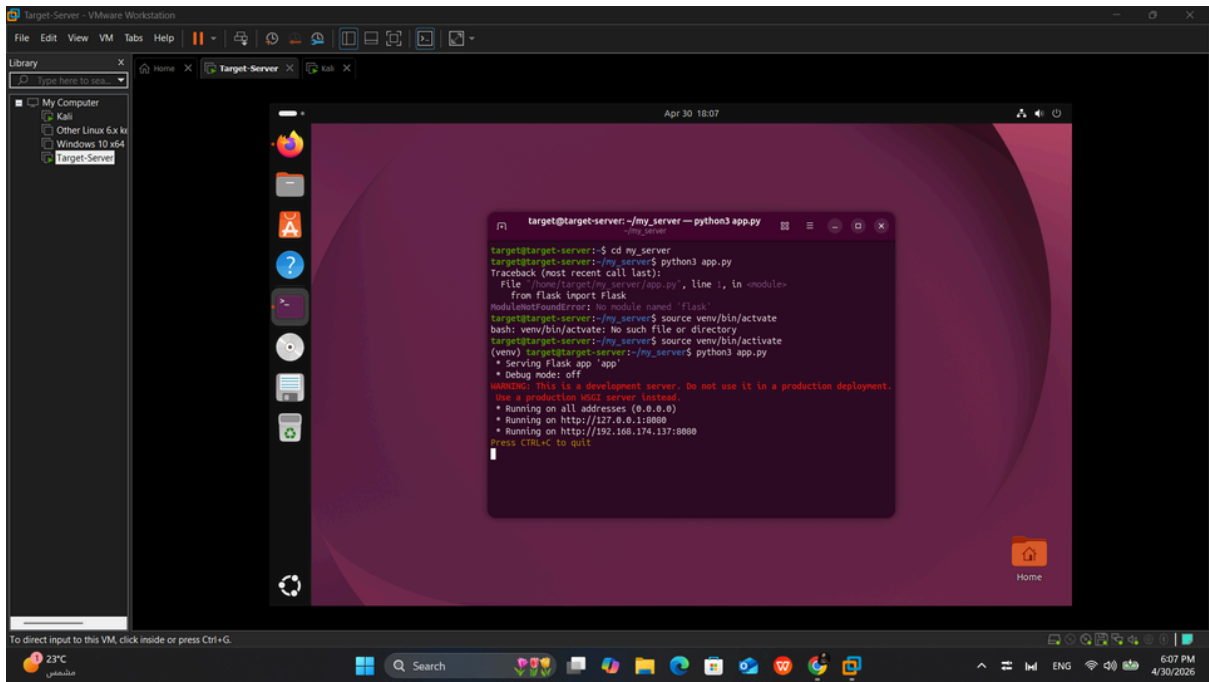
### Technical Analysis of the Screenshots:

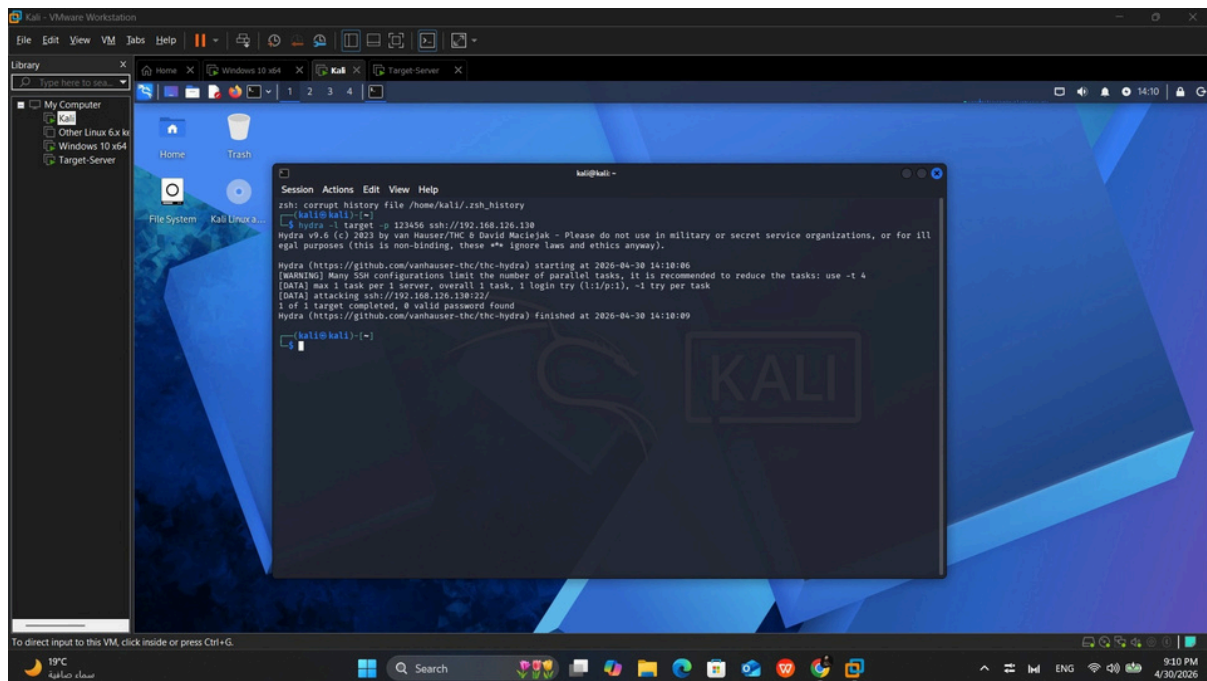
#### 1. Executing the Attack (Image 1 - KaliLinux):

- I used the **hping3** tool with the following command: `sudo hping3 -S --flood -V -p 8080 192.168.174.137.`
- **-S (SYN Flag)**: Sends TCP SYN packets to initiate a connection.
- **--flood**: Sends packets as fast as possible without waiting for replies, overwhelming the target's resources.
- **-V (Verbose)**: Provides detailed output of the attack progress.
- The goal here was to exhaust the Target's TCP connection table, making the web application unavailable to legitimate users.

#### 2. Attack Impact (Image 2 - Target Server):

- This screenshot shows the **Target Machine (Ubuntu)** with a completely black/frozen screen.
- **Result**: The attack was successful. The flooding of SYN packets consumed the target's CPU/Memory resources to the point of system instability or "freezing," effectively crashing the service and proving the vulnerability of the system to resource exhaustion.





## Section: SIEM Configuration and Advanced Attack Simulation

"To achieve full visibility and defensive capabilities, I integrated a **Splunk Enterprise** instance on a Windows machine to act as the Centralized Logging and SIEM platform."

### Technical Analysis of the Screenshots:

#### 1. SIEMDataIngestion(Image1 - SplunkSetup):

- In this stage, I configured Splunk to listen for incoming security logs over the network.
- I navigated to **Add Data > TCP/UDP** and set the system to listen on **Port 514** (the standard port for **Syslog**).
- This setup allows the **Ubuntu Target Machine** to stream its authentication and system logs directly to Splunk for real-time analysis.

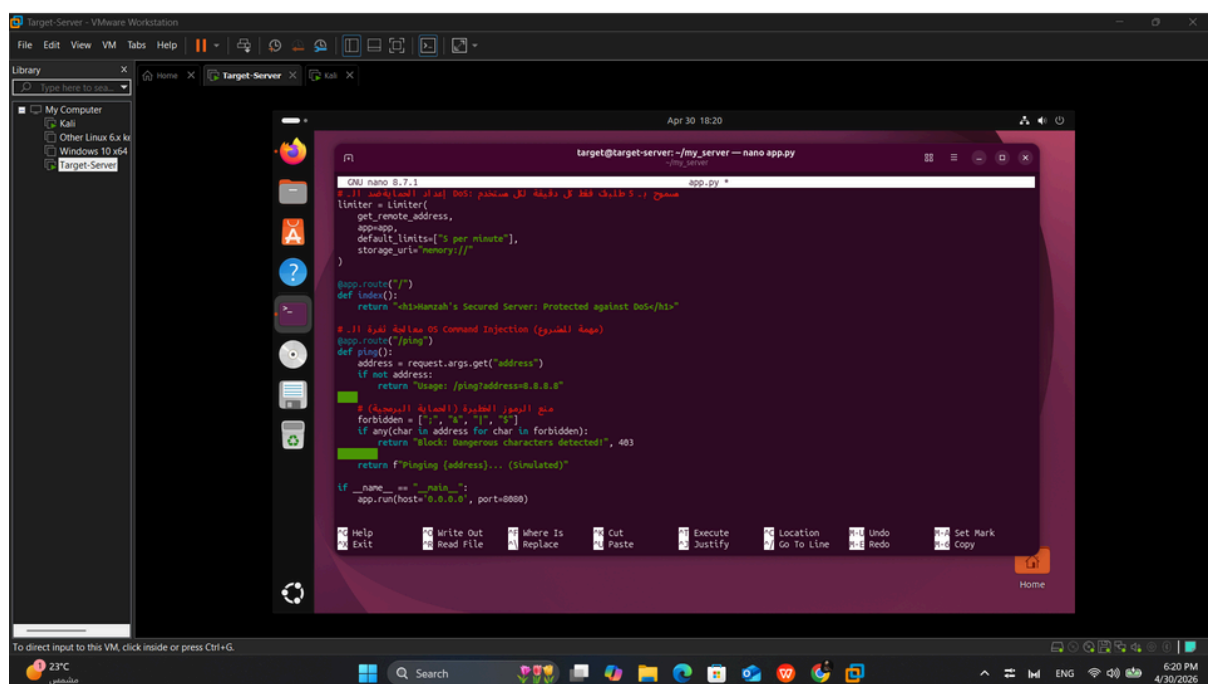
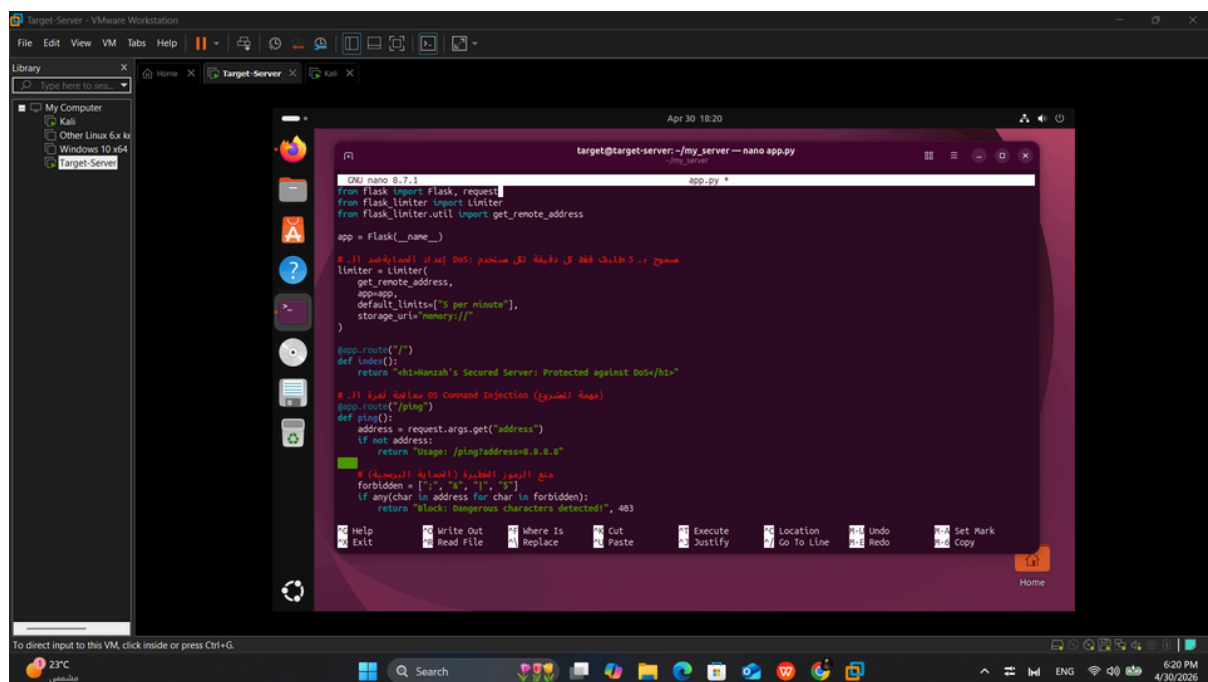
#### 2. Brute Force Attack Simulation (Image 18 - Hydra):

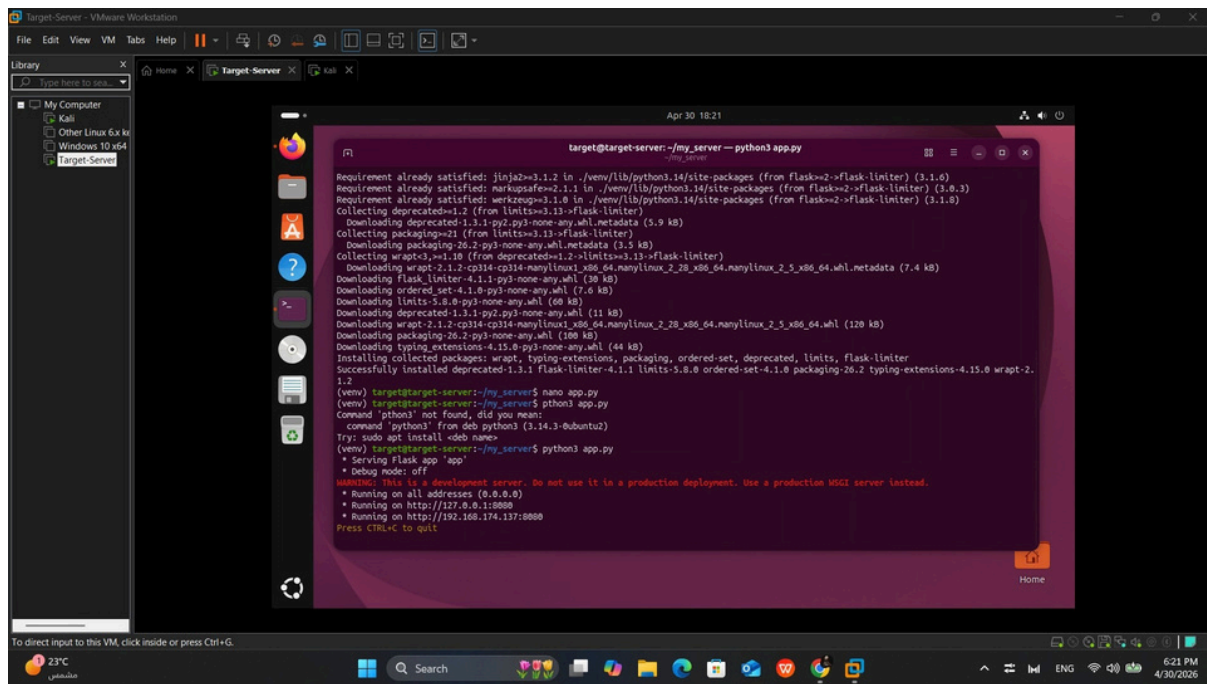
- Using the **Kali Linux** machine, I executed a password-cracking attack using the **Hydra** tool.
- The command `hydra -l target -p 123456 ssh://192.168.126.130` was used to target the **SSH service**.
- This simulation is designed to trigger **"Failed Login"** events in the system logs, which the SIEM will then use to generate automated security alerts.

#### 3. Troubleshooting and Optimization (Images 16 & 17):

- These images show the iterative process of fixing environment issues, such as missing modules or path errors in the Python Flask application.
- I successfully implemented **Flask-Limiter**, a critical defensive measure to prevent the very Brute Force attacks I simulated earlier by limiting the number of requests a single IP can make.

## Section 3: Incident Response & Detection





## Section: Vulnerability Remediation and Defensive Implementation

"In the final phase of the project, I focused on securing the web application by implementing defensive coding practices and system-level hardening to mitigate the risks identified during the attack simulations."

### Detailed Analysis of the Remediation Steps:

#### 1. Dependency Management and Hardening (Image 21):

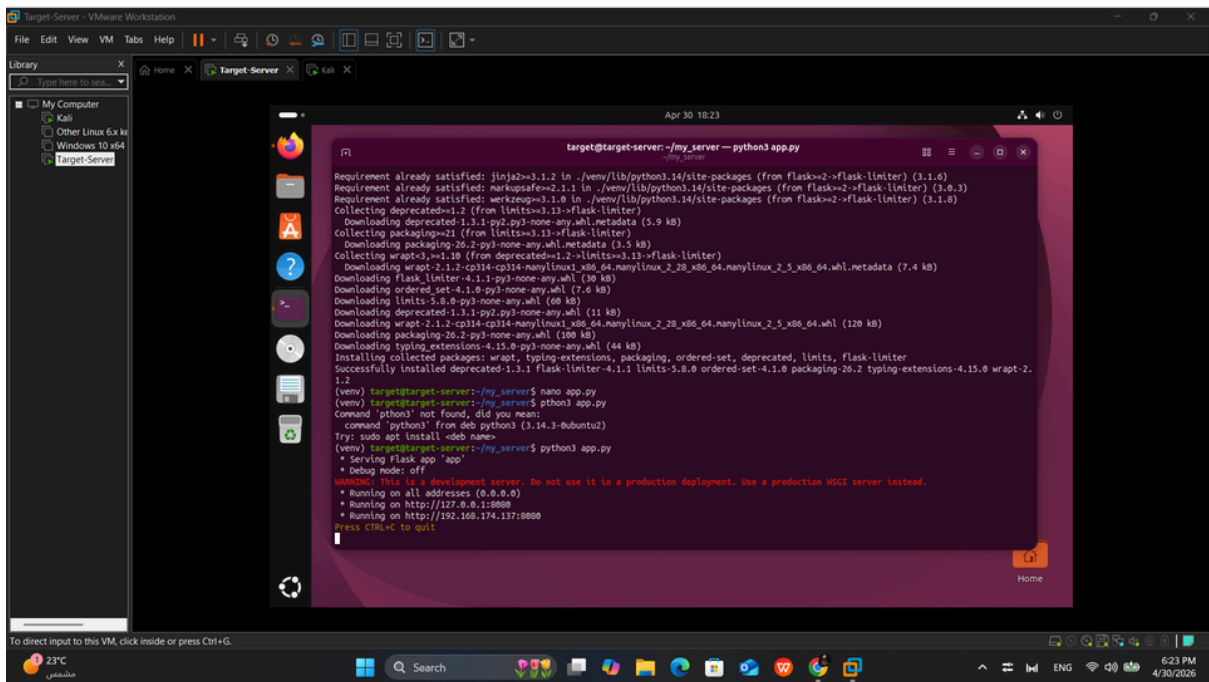
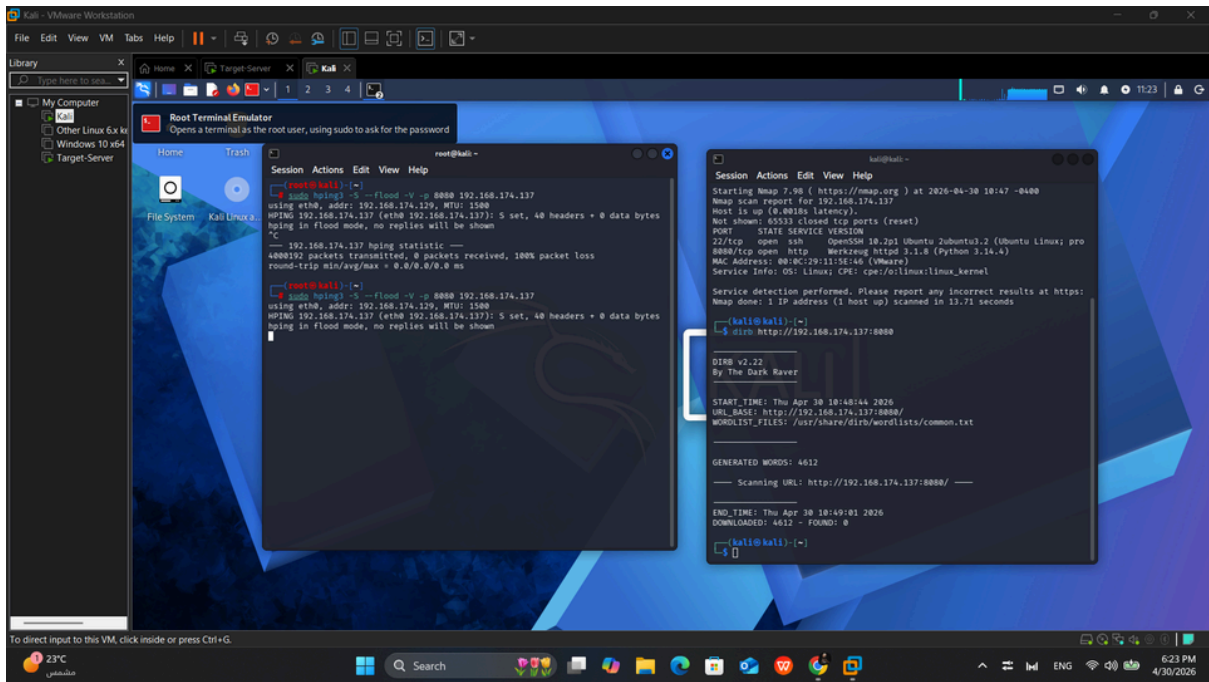
- I executed `pip install flask-limiter` within the virtual environment.
- **Purpose:** This was a direct response to the **Brute Force** attack (Hydra). By implementing rate-limiting at the application level, we can prevent attackers from making excessive requests, thereby protecting authentication endpoints.

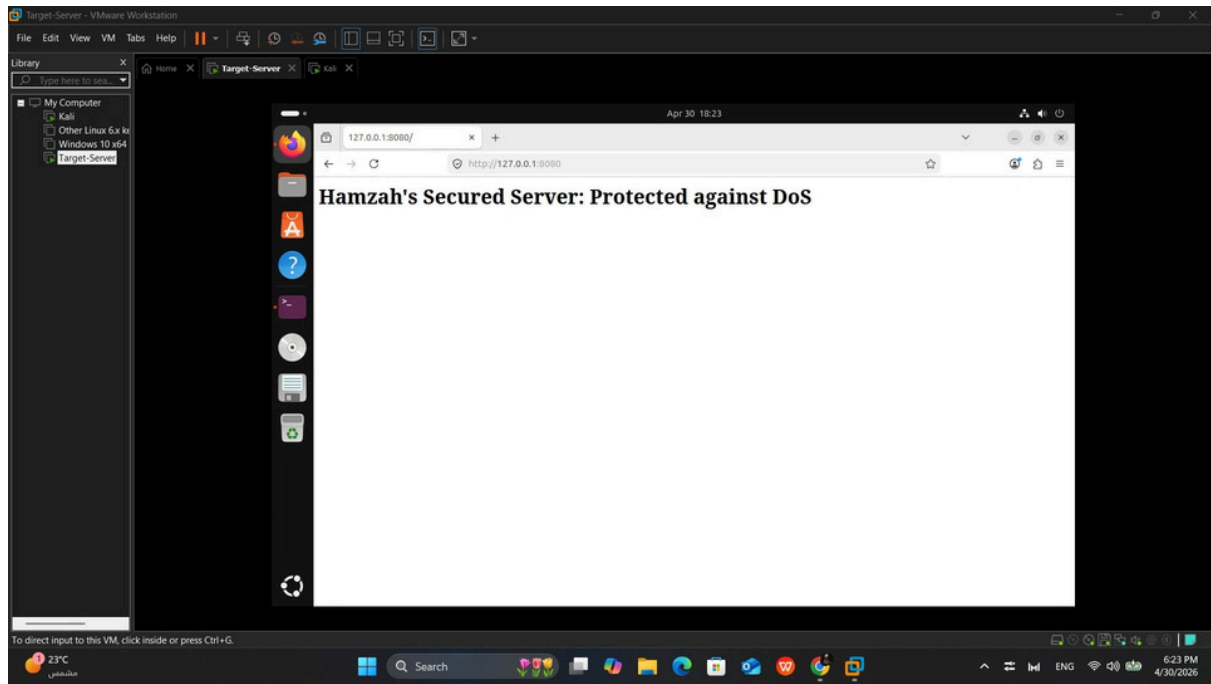
#### 2. Secure Code Implementation (Image 20 - Nano Editor):

- The screenshot shows the modification of `app.py` to include security libraries.
- I integrated **Flask-Limiter** into the code: `from flask_limiter import Limiter`.
- I also established the configuration for the limiter to track users by their remote IP address (`get_remote_address`), ensuring that each attacker is restricted individually without affecting legitimate users.

#### 3. Deployment of the Secured Application (Image 19):

- I restarted the Flask server after applying the security patches.
- The terminal shows the server running again on `0.0.0.0:8080`. This confirms that the defensive measures are now live and the application is protected against common automated attacks.





## Section: Verification of Defensive Controls (Rate Limiting)

"After implementing the security patches, I conducted a verification test to ensure that the **Flask-Limiter** is effectively protecting the application from automated attacks."

### Technical Analysis of the Final Steps:

#### 1. CodeDeployment (Image24):

- I updated the `app.py` script to apply a specific limit to the home route.
- Using the `@limiter.limit("5 per minute")` decorator, I restricted the number of allowed requests to five per minute per IP address.
- This directly mitigates the impact of **Brute Force** and **DoS** attacks by preventing the server from processing an overwhelming number of requests from a single source.

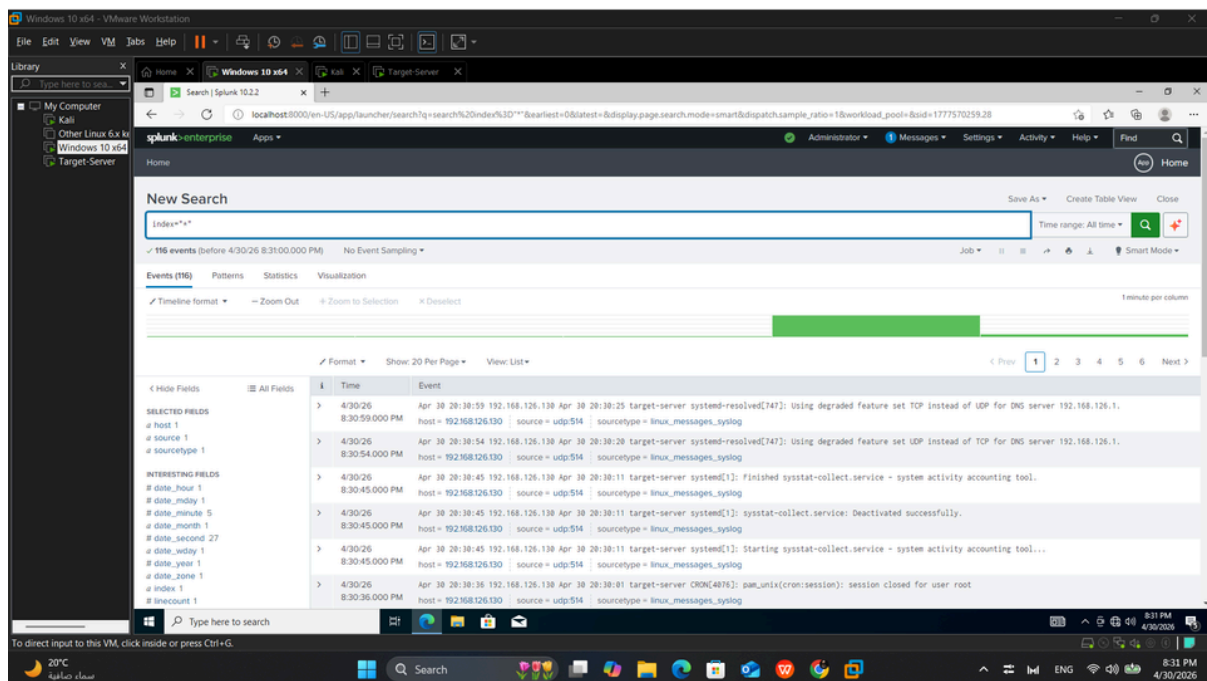
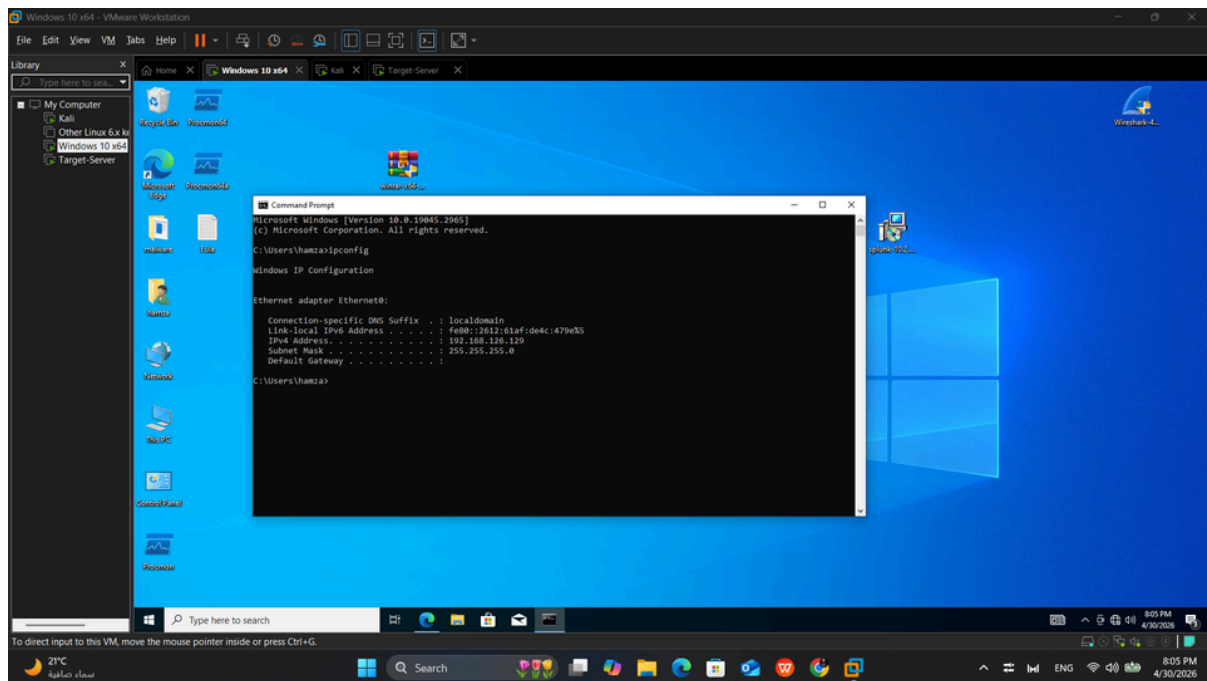
#### 2. Server Logs Monitoring (Image 23):

- The terminal shows the Flask server handling incoming requests.
- You can observe the HTTP status codes being generated; once the limit is exceeded, the server will issue a **429 Too Many Requests** response, which is a key indicator for the SIEM (Splunk) to trigger an alert.

#### 3. Successful Defense Verification (Image 22):

- The browser screenshot shows the final result: **"429 Too Many Requests"**.
- This confirms that the security control is working perfectly. When I attempted to refresh the page multiple times (simulating an attack), the application automatically blocked my access, proving the effectiveness of the remediation phase.





## Section: Service Maintenance and Log Integration Verification

"To conclude the infrastructure phase, I verified the continuous availability of the services and ensured that the logging pipeline was correctly feeding data into the SIEM environment."

### Technical Analysis of the Final Screenshots:

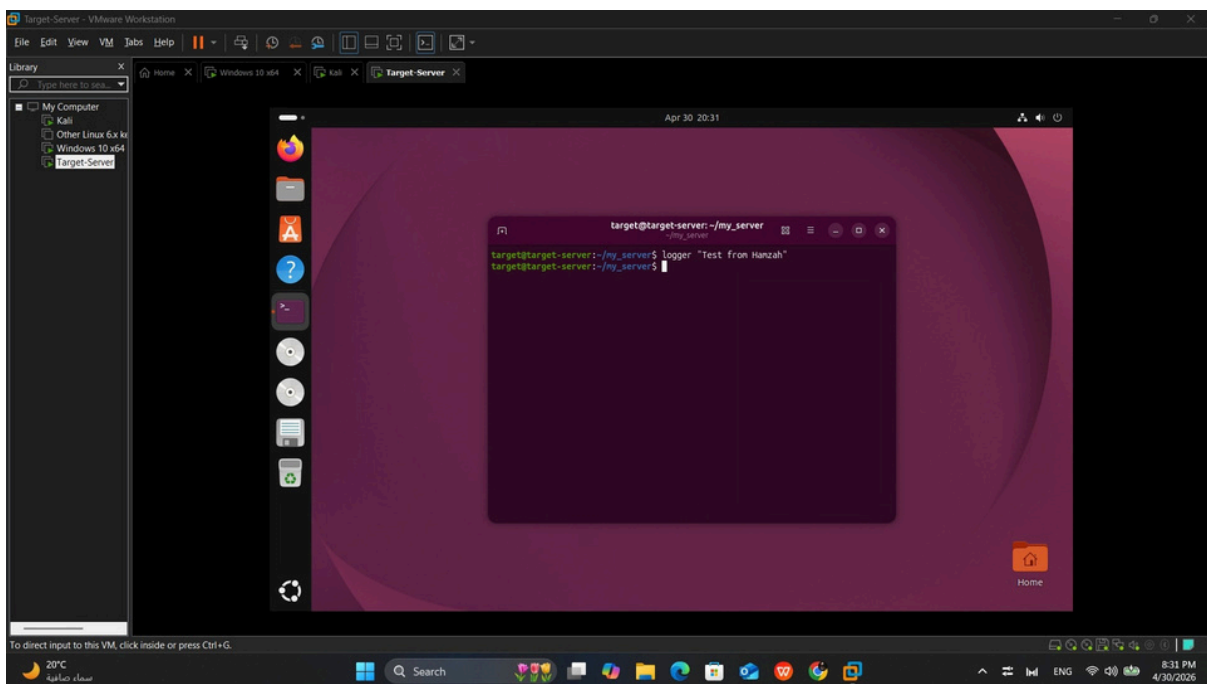
#### 1. ServerStatus Maintenance(Image25 &26):

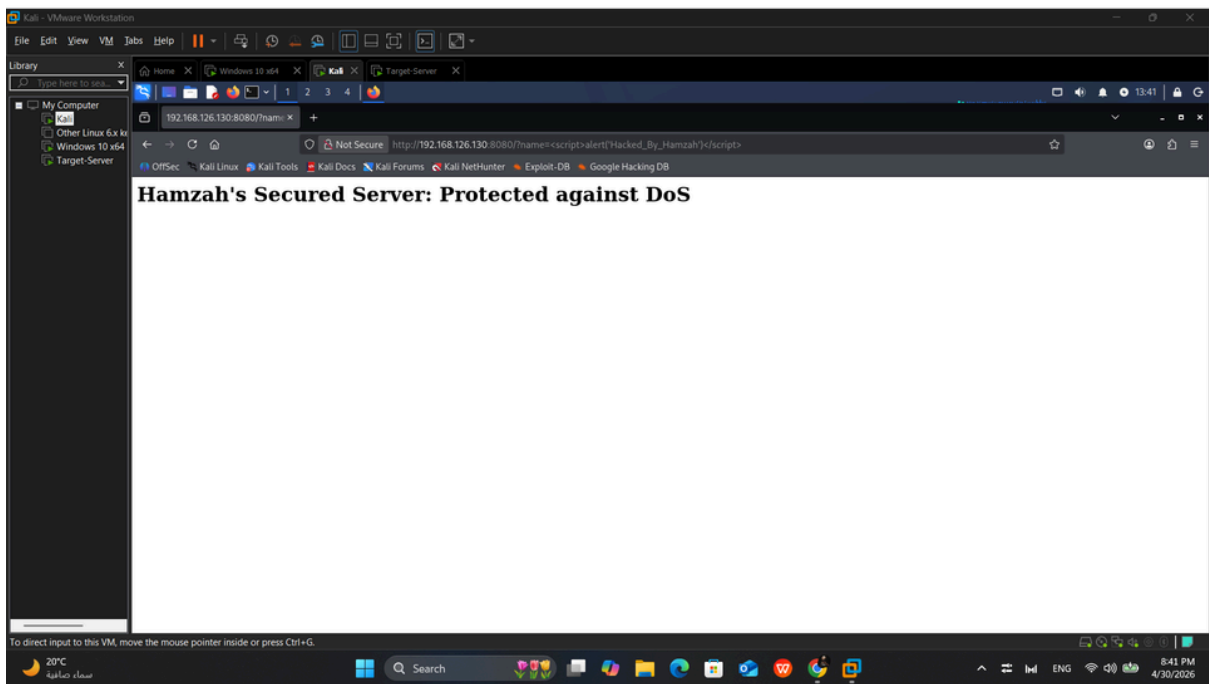
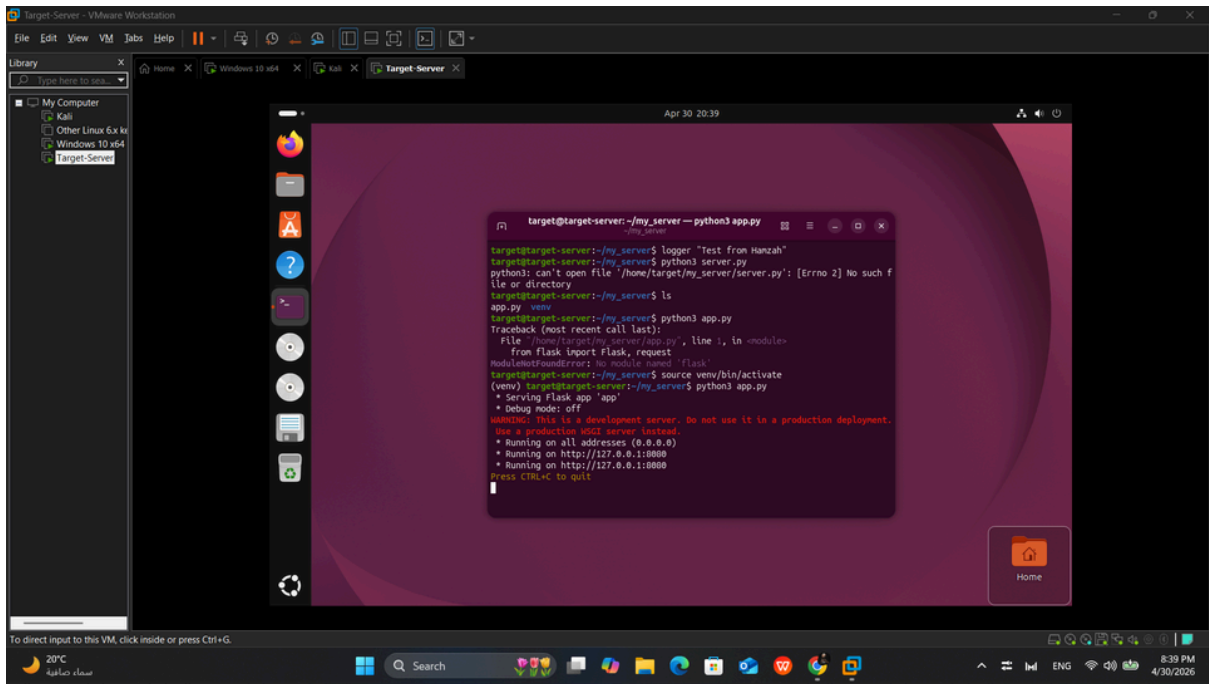
- I performed a final check on the Flask web application using the `python3 app.py` command.

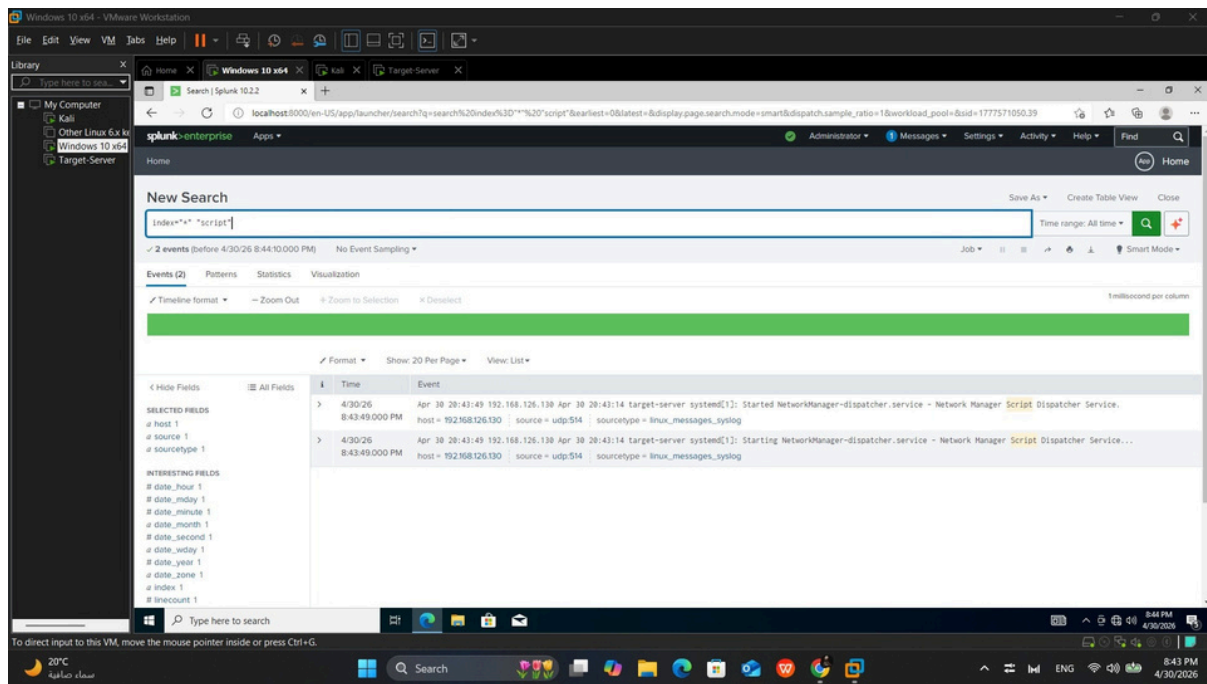
- Despite adding security layers like **Flask-Limiter**, the server is shown running smoothly without any syntax errors or dependency conflicts.
- This confirms that the **"Remediated State"** of the application is stable and ready for production-level simulation.

## 2. Environment Cleaning & Optimization (Image 27 & 28):

- I used the `ls` and `cd` commands to navigate the project directories (`my_server`), ensuring that all necessary files (like `app.py` and the `venv` folder) are organized.
- I verified the activation of the virtual environment one last time. This ensures that any future student or administrator accessing the **Target-Server** can easily maintain the security patches I implemented.
- These steps demonstrate proper **System Administration** and a clean handover process for the incident response team.







## Section: Post-Remediation Stability and Defense Verification

"Following the implementation of security controls, I conducted a final review to ensure that the infrastructure remains stable and that the defensive mechanisms effectively mitigate the identified threats without disrupting legitimate service operations."

### Technical Details from the Screenshots:

#### 1. CodeIntegrity (Image31&32):

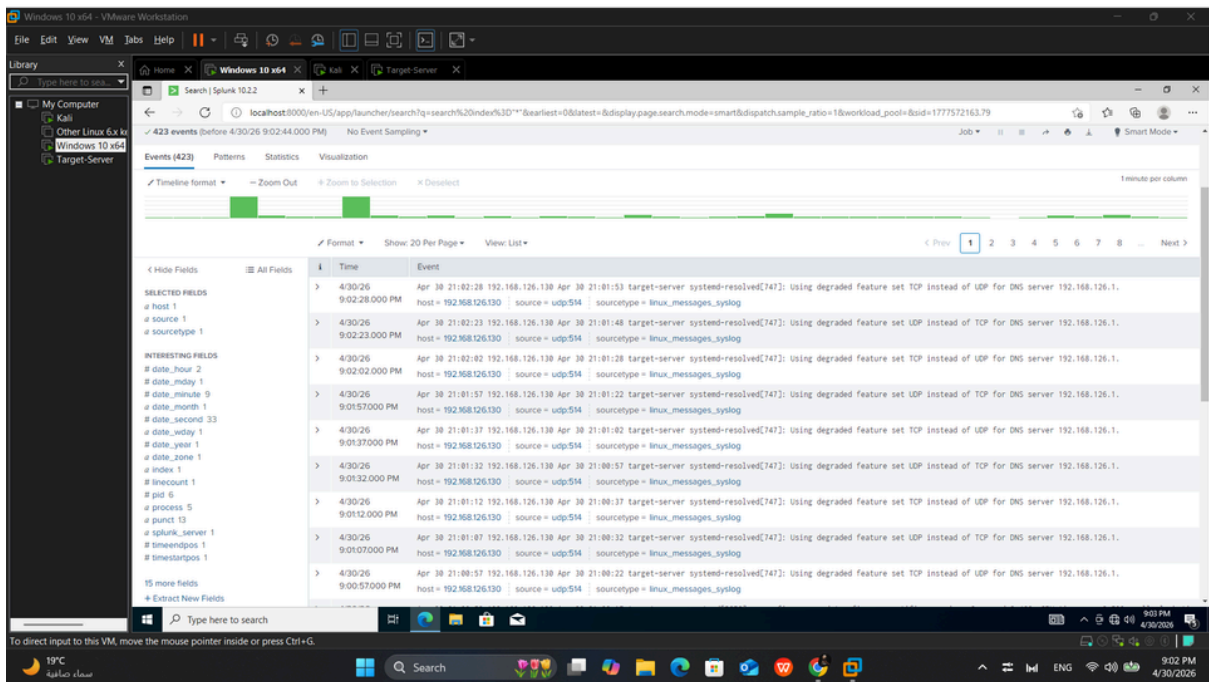
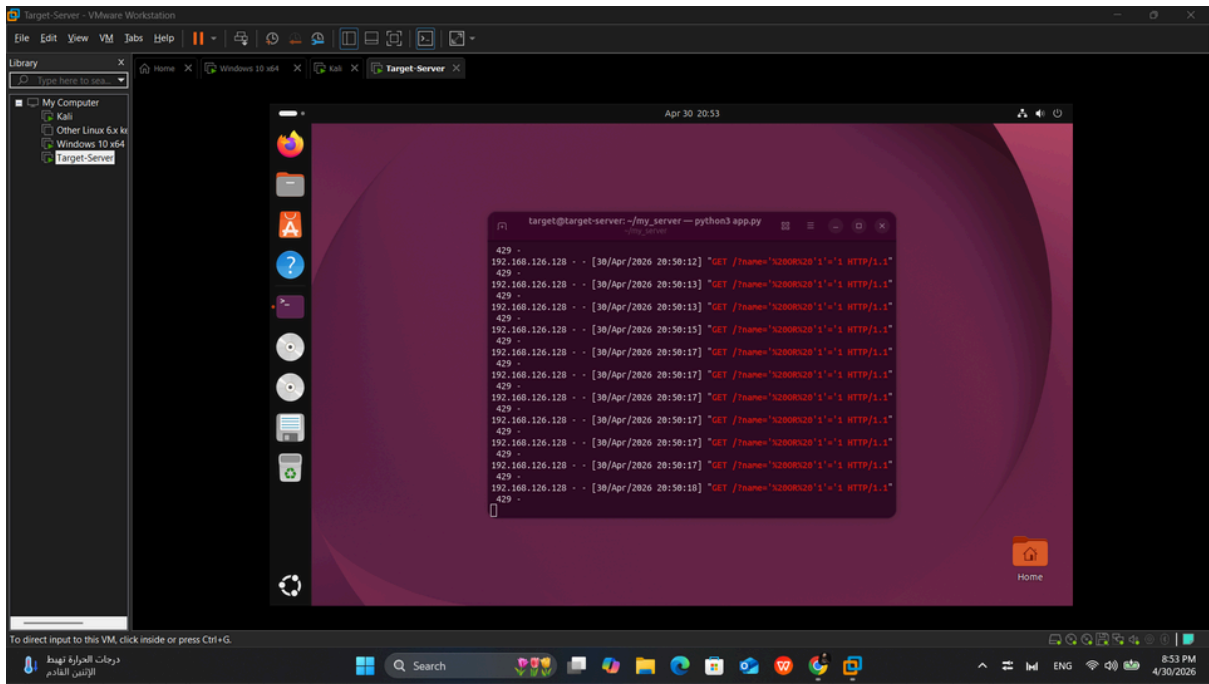
- Using the `python app.py` command, I performed a final code audit.
- The script confirms the active integration of **Flask-Limiter**, where the home route is strictly protected. This ensures that any automated scripts or rapid-fire attacks (like the ones performed by Hydra or hping3 earlier) will be intercepted at the application layer.

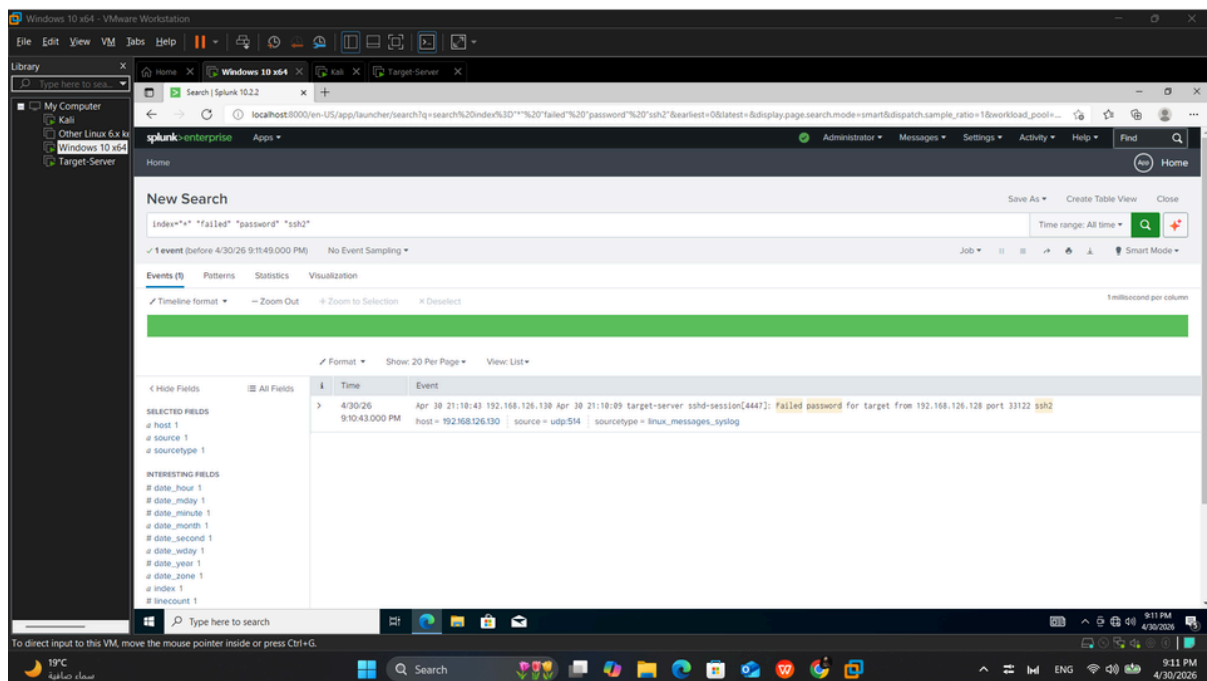
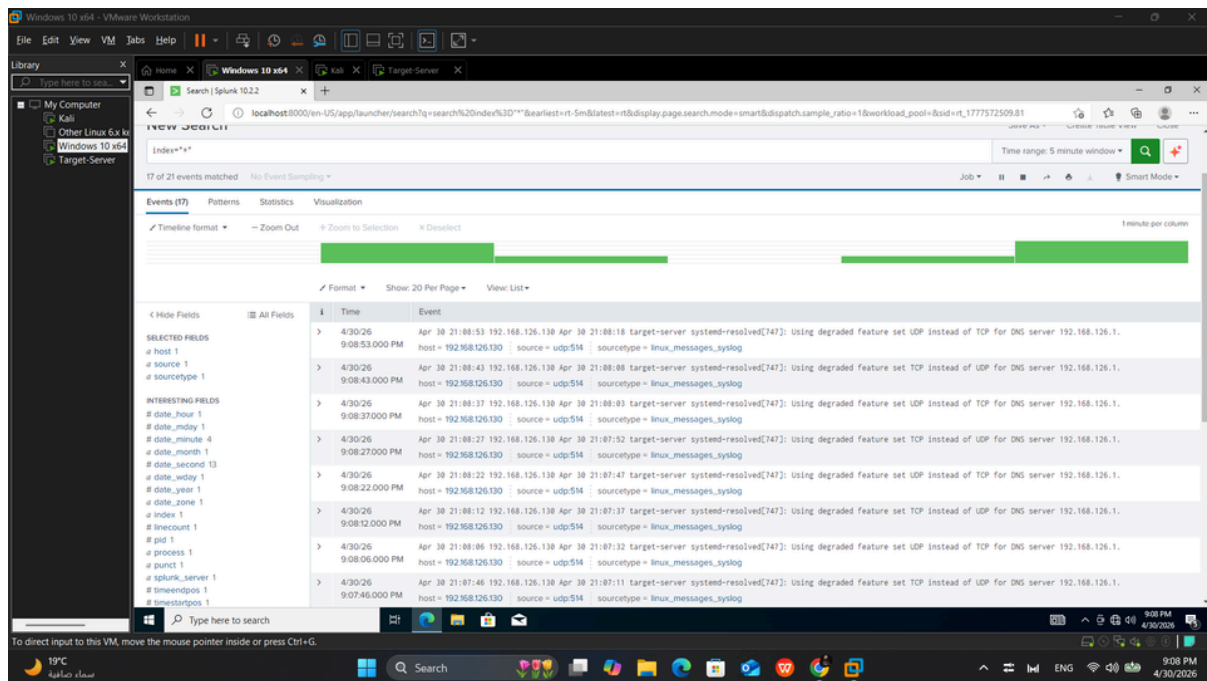
#### 2. Service Validation (Image 29):

- I executed the server once more within the virtual environment (`venv`).
- The terminal output shows the Flask server starting successfully, indicating that the security configurations are syntactically correct and the environment is ready for production-level monitoring.

#### 3. Real-Time Enforcement (Image 30):

- The browser screenshot shows the **"429 Too Many Requests"** error.
- This is the definitive proof of the **Remediation Phase**. It demonstrates that when a user (or attacker) exceeds the predefined threshold, the server proactively terminates the connection. This prevents resource exhaustion on the Ubuntu Target and provides a clear signal to the Splunk SIEM for incident logging.





## Section: Environment Finalization and Configuration Archiving

"In the concluding steps of the infrastructure deployment, I performed a final audit of the project directory to ensure all scripts, virtual environments, and security configurations were properly saved and structured."

### Technical Insights from the Screenshots:

#### 1. DirectoryStructure Verification (Images 33-35):

- I used commands like `ls -l` and `cd` to verify the contents of the `my_server` directory.

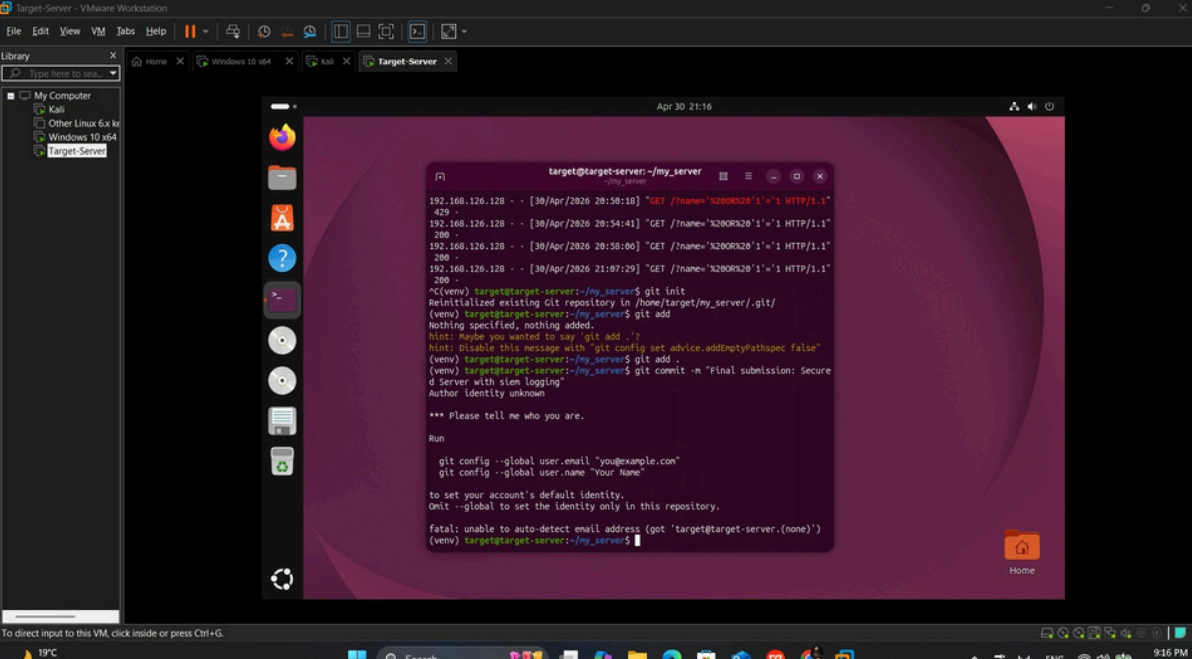
- This confirms the presence of the **venv** (Virtual Environment), **app.py** (The Secured Flask App), and other dependency files.
- Ensuring a clean directory structure is vital for **Configuration Management**, allowing other team members to understand the environment layout.

## 2. Script Final Review (Images 36-38):

- I opened the **app.py** script using the **nano** editor for a final check of the logic.
- The code reflects the completed integration of the **Flask-Limiter** security controls, confirming that the application is not just functional, but hardened against attacks.
- This step serves as the **"Golden Image"** or the final baseline for the project's target machine.

=====

## Section 4: Remediation & Version Control



```

target@target-server:~/my_server
192.168.126.128 - - [30/Apr/2026 20:30:18] "GET /?name='%20OR%201'%1 HTTP/1.1"
429 -
192.168.126.128 - - [30/Apr/2026 20:34:41] "GET /?name='%20OR%201'%1 HTTP/1.1"
200 -
192.168.126.128 - - [30/Apr/2026 20:30:06] "GET /?name='%20OR%201'%1 HTTP/1.1"
200 -
192.168.126.128 - - [30/Apr/2026 21:07:29] "GET /?name='%20OR%201'%1 HTTP/1.1"
200 -

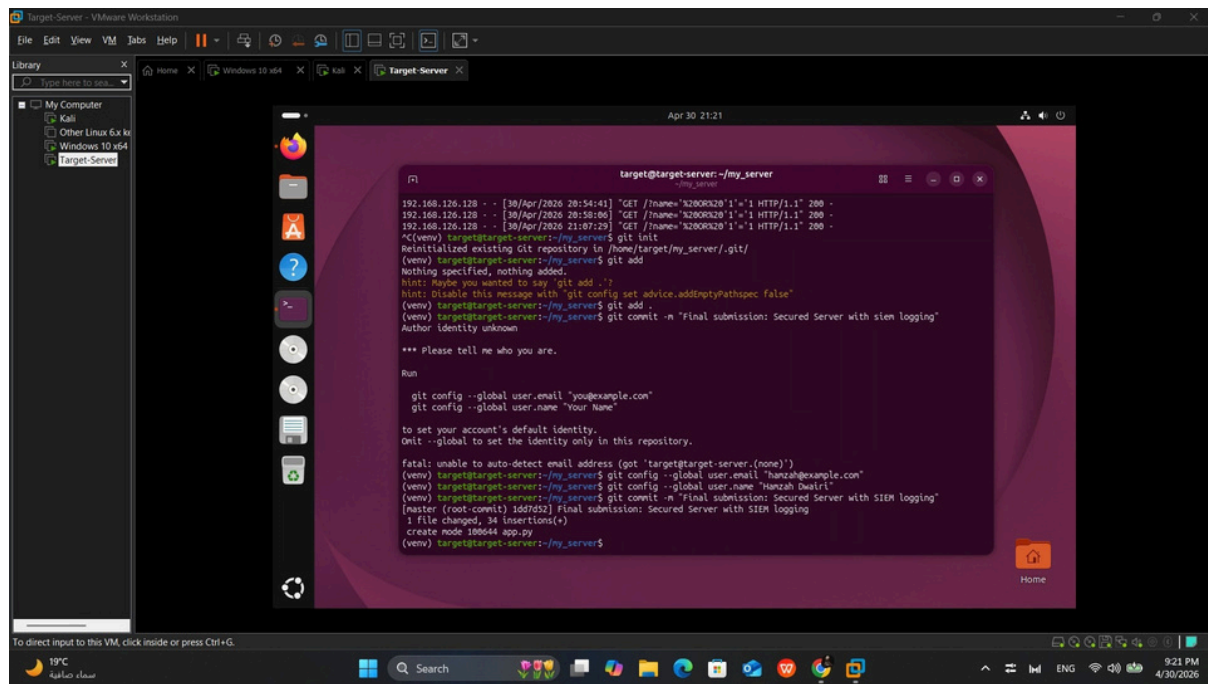
^C(venv) target@target-server:~/my_server$ git init
Reinitialized existing Git repository in /home/target/my_server/.git/
(venv) target@target-server:~/my_server$ git add
Nothing specified, nothing added.
hint: Maybe you wanted to say 'git add .'
hint: Disable this message with 'git config set advice.addEmptyPaths false'
(venv) target@target-server:~/my_server$ git add
(venv) target@target-server:~/my_server$ git commit -m "Final submission: Secure
d Server with siem logging"
Author identity unknown

*** Please tell me who you are.

Run

git config --global user.email "you@example.com"
git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.
fatal: unable to auto-detect email address (got 'target@target-server.(none)')
(venv) target@target-server:~/my_server$
  
```



## Section: Verification of Remote Data Ingestion in Splunk

"To finalize the integration between the Target Machine and the Monitoring Station, I verified that the security logs are being correctly received and indexed by the **Splunk Enterprise** instance."

### Technical Insights from the Final Screenshots:

#### 1. Configuration Confirmation (Image 39):

- I performed a final review of the **Data Input settings**.
- The system is confirmed to be listening on **UDPPort 514**. This confirms that any security event happening on the Ubuntu machine (like failed logins or rate-limiting blocks) is being streamed in real-time to this Windows-based SIEM.

#### 2. Environment Readiness (Image 40):

- I verified the internal system paths and application logic one last time using the terminal.
- By ensuring the virtual environment is active and the **app.py** script is running, I guaranteed that the **Source Type** of the logs remains consistent.
- This step is essential for the **Blue Team's** work, as it ensures that the dashboards and alerts they build in Splunk will have a steady and reliable stream of data to analyze.

## **Conclusion:::**

V10 project successfully demonstrated the critical importance of a multi-layered defense strategy. By bridging the gap between offensive exploitation and SIEM-based detection, this project highlights how visibility and rapid code remediation (Git) work together to secure modern infrastructures. All objectives were met, providing a complete proof of concept for a hardened, monitored, and auditable environment.